



Machine Learning Technique for Enhanced Situational Awareness With Fire Fighter Drones

A graduation Project submitted to the faculty of artificial intelligence in partial fulfillment of the requirements for the degree of Bachelor of Science in artificial intelligence

By

Ibrahim Mohamed Ibrahim

Hassan Ali Hassan

Ali Atef Abu al Yazid

Seif El-Din Ayman

Nada Emad El-din El-katan

Manal gamil Alaa El-din

Rahma Mohamed Abdul Majed

Fatma Tarek Mohamed

Supervised By

Dr. Zeinab Hassan Ali Hassan

**Assistant Professor at Embedded and Network Technology Department, Faculty
of Artificial Intelligence, KFS University**

Faculty Of Artificial Intelligence

Kafr EL -Sheikh University

2023 – 2024

DEDICATION

We would like to express our heartfelt gratitude and dedicate our graduation project robot to several individuals who have played a significant role in its success.

First, we extend our appreciation to Dr. zainab Hassan for her guidance, support and encouragement throughout this project.

We would also like to extend our gratitude to our team members, consisting of 8 individuals who have contributed to make this project a reality

Additionally, we would like to express our thanks to our families to those who were our source of support and giving and our friends to those who are delighted by our success, who smoothed out the pitfalls of our path, and who stand behind us like a shadow, no matter how many flops we make.

Lastly, we extend our appreciation to the entire faculty and staff of our institution.

We place this project in your hands, hoping that it will receive your good satisfaction.

ABSTRACT

The main objective of the project is to propose the design and development of a fire detection and monitoring drone system to aid in early fire detection, assessment, and response. the proposed system comprises a custom-built drone equipped with a high-resolution camera and, optionally, a thermal imaging camera for enhanced fire detection capabilities. The drone is integrated with a GPS module for autonomous navigation and location tracking.

Additionally, the drone is equipped with a communication module for real-time data transmission to a ground control station. the software component of the system includes computer vision algorithms for image processing and fire detection. These algorithms analyze live video feeds from the drone's cameras to detect signs of fire, such as flames. Upon detecting a fire, the system can alert emergency responders and provide them with real-time data, including the fire's location and size. the system's effectiveness and performance will be evaluated through simulations and real-world tests.

The project aims to demonstrate the feasibility and practicality of using drones for fire detection and monitoring, highlighting their potential to improve the efficiency and effectiveness of firefighting and emergency response operations. Drones have been increasingly used in firefighting to improve the response speed and reduce the dangers to human firefighters. However, few studies simultaneously consider fire spread prediction, drone scheduling, and the configuration of supporting staff and supplies.

Keywords: Deep Learning; Fire Detection; Smoke Detection; Emergency Response; Unmanned Aerial Vehicle (UAV); Situational Awareness

TABLE OF CONTENTS

DEDICATION	2
LIST OF TABLES	9
LIST OF FIGURES	10
CHAPTER 1: INTRODUCTION	
1.1 Overview	11
1.2 Problem Statement	11
1.3 Objective	11
1.4 Significance	11
1.5 Scope	12
1.6 Motivation	12
1.7 Contributions	13
1.8 Organization	13
Chapter 2: Background & Related Work	
2.1. Overview	15
2.2. Background	16
2.2.1. Historical Context	16
2.2.2. Early Adoption	16
2.2.3. Technological Adv	16

TABLE OF CONTENTS

2.2.4. Use Cases	17
2.2.5 Challenges and Future Directions	18
2.3. Related Work	18
2.3.1. Fire Fighting Drone Using Extinguisher Bomb	19
2.3.1.1. Design and Proposed Features	19
2.3.1.2. Features	20
2.3.1.3. Conclusion	20
2.3.2. Firefighter Drone by Computer Engineering Department	20
2.3.2.1. Design and Proposed Features	21
2.3.2.3. Conclusion	21
2.3.3. Firefighting Drone Configuration and Scheduling for Wildfire Based on Loss Estimation and Minimization	22
2.3.3.1. Design/Proposal	23
2.3.3.2. Features	23
2.3.3.3. Conclusion	23
2.3.4. Enhancing Firefighting Efficiency: Manual ESC equipped Quadcopters with KK 2.1.5 Flight Controller	23
2.3.4.1. Design/Proposed System	24
2.3.4.2. Features	24
2.3.4.3. Conclusion	25
2.3.5. Use of drones for firefighting operations	25
2.3.5.1. Design/Proposed	26
2.3.5.2. Features	26
2.3.5.3. Conclusion	26
2.3.5.4. Future Scope	26
2.4. Summary	27

CHAPTER 3: SIMULATION

3.1. Overview	28
3.2. The main components of simulation	28
3.2.1 Gazebo	28
3.2.2 Ros	29
CHAPTER 4: System Architecture	
4.1. Overview	49
4.2 Hardware	50
4.2.1. APM Wiring Diagram	51
4.2.2. Raspberry pi	53
4.2.3. camera 1.3v	54
4.2.4. ESC 30A	55
4.2.5. Brushless Motor	56
4.2.6. propellers	57
4.2.6. Dual-side prototype	58
4.2.7. Buck converter	59
4.2.8. LIPO battery	60
4.2.9. f550 hexa-copter frame	61
4.3 Implementation of hardware	62
4.4 circuits	63
CHAPTER 5: Machine Learning Algorithm	
5. 1 Overview	64
5.2 Pre Processing the data	65
5.2.1. preparing the dataset	65
5.2.1.1. data gathering	65
5.2.1.2. Annotation	65
5.2.1.3. Data augmentation	65
5.2.1.4. Roboflow	66
5.3. Why YOLO?	66
5.3.1. Purpose and Scope	66
5.3.1. Purpose and Scope	66
5.4. YOLO	67
5.4.1. Usage	67
5.5. YOLO Algorithm	67
5.5.1. YOLO Architecture	67
5.5.2 How YOLO works?	68
5.5.3 YOLOv8	69
5.6. IDE	70
5.6.1. Jupiter Notebook	70
5.6.2. Google Colab	71

5.6.3. Programming	72
5.6.4. libraries	72
5.6.4.1 OpenCV	72
5.6.4.2. Ultralytics	74
5.7. Distance Calculation	78
CHAPTER 6: linking between FCU and Raspberry Pi	
6.1 Overview	80
6.2. Component	82
6.2.1. APM in Flight Controllers	84
6.2.2 Reasons Why APM 2.8 is Considered Outdated	84
6.3. Link between raspberry pi and apm	84
6.3.1 Power Connections	84
6.3.2 UART	85
6.3.3 UART Serial communication	85
CHAPTER 7: CONCLUSION	92
7.1. Conclusion and limitations	93
7.2. FUTURE WORK	94
REFERENCES	95

LIST OF TABLES

Table 1: hardware component.....	29
Table 2: hardware component2.....	64

LIST OF FIGURES

Figure 1: Work space 1	42
Figure2: Drone simulation	49
Figure 3: APM wiring	52
Figure 4: APM 2.8	53
Figure 5:Raspberry pi 4 4GB	54
Figure 6:5MP camera v1.3	55
Figure 7:ESC 30A	56
Figure 8:Brushless motor 2400kv	57
Figure 9:Propellers 10x4.7	58
Figure 10: Dual-side prototype PCB	59
Figure 11:Buck converter 3A	60
Figure 12: 3S lipo battery 5200mAh	61
Figure 13:3D printed f550 hexa-copter frame	62
Figure 14:Shows the drone	63
Figure 15:The bottom panel	64
Figure 16:Roboflow	67
Figure 17:CNN	68
Figure 18:YOLO	69
Figure 19: Jupyter-dashbord	71
Figure 20:Jupyter notebook	72
Figure 21:Matlab	74
Figure 22:Importing Ultralytics and roboflow	76
Figure 23:Lunching data from Roboflow to Colab	76
Figure 24:Adam optimizer	77
Figure 25:The result	77
Figure 26:Final result of detection	78
Figure 27:Validation and testing	79
Figure 28:The equation of distance	79
Figure 29: Raspberry pi	84
Figure 30:Zutopilot apm 2.8	84

CHAPTER 1

INTRODUCTION

CHAPTER 1: INTRODUCTION

1.1 Overview

In recent years, the frequency and intensity of wildfires have increased dramatically, posing significant threats to lives, property, and ecosystems. Traditional firefighting methods, while effective, often place firefighters in hazardous situations and can be limited in their reach and responsiveness. To address these challenges, technological innovations are essential.

1.2 Problem Statement

The primary challenge in firefighting is the rapid detection and suppression of fires, especially in inaccessible or dangerous areas. Current firefighting techniques can be slow and resource-intensive, risking the lives of firefighters and causing extensive damage before the fire is brought under control.

1.3 Objective

This project aims to develop an autonomous firefighting drone capable of detecting, assessing, and suppressing fires with minimal human intervention. The drone is designed to operate in environments that are too dangerous or difficult for human firefighters to reach, providing a safer and more efficient solution for firefighting efforts.

1.4 Significance

The development of an autonomous firefighting drone represents a significant advancement in emergency response technology. By leveraging artificial intelligence, robotics, and advanced sensor technology, this project seeks to enhance the capabilities of firefighting teams, reduce response times, and minimize the risks associated with firefighting operations.

1.5 Scope

The project encompasses the design, development, and testing of a prototype autonomous firefighting drone. Key components include:

- i) Fire Detection System: Utilizing thermal imaging and smoke detection sensors to identify and locate fires accurately with help of computer vision using YOLOv8.

- ii) Navigation and Control: Implementing advanced algorithms for autonomous navigation, obstacle avoidance, and path planning.
- iii) Fire Suppression Mechanism: Developing an effective fire suppression system, such as water or fire-retardant dispensers, integrated into the drone.
- iv) Communication and Coordination: Ensuring seamless communication between the drone, control stations, and other firefighting units.

1.6 Motivation

The increase in the frequency and severity of fires globally has led to significant losses, highlighting the need for advanced firefighting solutions. Firefighting is dangerous, often putting firefighters in hazardous situations. Autonomous firefighting drones can reduce human risk by handling the most dangerous tasks.

Advancements in robotics, AI, and sensors present new opportunities for innovative firefighting solutions. Drones can offer mobility, flexibility, and the ability to operate in harsh conditions. Wildfires cause severe environmental damage. Improved fire detection and suppression through drones can help mitigate these impacts and preserve natural habitats.

This project aligns with personal interests in robotics, AI, and disaster management, providing a practical application for academic knowledge and fostering innovation.

Autonomous firefighting drones can revolutionize firefighting practices, enhancing response times, reducing risks to human life, and minimizing damage, thereby benefiting communities worldwide.

1.7 Contributions

The novelty of the project called “Fire Detection System” can be organized as follows:

- i) This project introduces an advanced fire detection system utilizing the YOLOv8 computer vision model. This model can accurately detect and track fires in real-time, even in challenging environments.
- ii) Autonomous Navigation and Control We developed sophisticated algorithms for autonomous navigation, enabling the drone to maneuver through complex terrains, avoid obstacles, and optimize its path to the fire source. Integrated Fire Suppression Mechanism A key innovation is the integration of an effective fire suppression system within the drone. This includes a mechanism for dispensing water or fire retardants, allowing the drone to actively combat fires upon detection.
- iii) Enhanced Communication and Coordination: The project includes a robust communication framework that ensures seamless interaction between the drone, control stations, and other firefighting units. This enhances coordination and efficiency during firefighting operations.
- iv) Real-World Testing and Validation: We conducted extensive real-world testing to validate the performance and reliability of the autonomous firefighting drone. These tests demonstrated the drone’s capability to operate in various fire scenarios, providing valuable data for further improvements.
- v) Open-Source Framework: The project contributes to the academic and research community by providing an open-source framework for autonomous firefighting drones. This framework can be used as a foundation for further research and development in the field.
- vi) Potential for Broader Applications While focused on firefighting, the technologies and methodologies developed in this project have potential applications in other fields, such as search and rescue operations, environmental monitoring, and disaster response.

1.8 Organization

This document is structured to provide a comprehensive overview of the development and implementation of the Autonomous Firefighting Drone. The thesis is organized in seven chapters, which can be presented as follows:

Chapter 2: Background and Related work

This chapter discusses the current challenges faced in firefighting, including risks to firefighters, difficulties in reaching remote or hazardous areas, and the need for rapid response.

And we will highlight the advancements in robotics, AI, and sensor technology that make autonomous firefighting drones feasible, Review current firefighting technologies and methods, Gaps and Opportunities.

Chapter 3: Simulation Using ROS

Purpose of Simulation why simulation is essential for the development and testing of the autonomous firefighting drone. We will discuss the benefits of using simulation before real-world testing, such as safety, cost-effectiveness, and the ability to test in various scenarios.

Chapter 4: FIRE DRONE SYSTEM

This chapter provides an overview of hardware components in the development of the autonomous firefighting drone like drone frame, motors, ESCs, sensors and camera, testing and calibration.

Chapter 5: Computer Vision

Overview of the role of computer vision in the project, emphasizing its significance in fire detection and tracking explain the choice of YOLOv8 for this project, Dataset Collection and Preparation, Model Training, Model Evaluation and Performance, Integration with the Drone System, Testing in Simulated and Real Environments, Challenges and Limitations and Future Improvements.

CHAPTER 6: linking between FCU and Raspberry Pi

This chapter describes the linking between a Flight Control Unit (FCU) and a Raspberry Pi can enable powerful integration and control capabilities in various applications. The FCU, commonly used in drones or unmanned aerial vehicles (UAVs), is responsible for managing flight control parameters and sensor data. On the other hand, the Raspberry Pi is a versatile single-board computer known for its computational capabilities and extensive peripheral support. By establishing a connection between the FCU and Raspberry Pi, it becomes possible to leverage the Pi's processing power and connectivity options to enhance the functionality of the FCU. This integration allows for real-time data analysis, advanced control algorithms, and the ability to interface with additional sensors or devices. Whether it's implementing computer vision algorithms for autonomous flight, integrating external sensors for environmental monitoring, or enabling wireless communication and data logging capabilities, the linking between an FCU and Raspberry Pi opens a world of possibilities for creating sophisticated and intelligent UAV systems.

Chapter 7: Conclusion and Future Work

Summary of the project, highlighting its objectives, methodologies, and key findings. Recap the contributions made by the project, including advancements in autonomous firefighting technology. Technical Challenges, Implications and Impact, Future Work of Enhancements to Existing Systems and Integration of AI Technologies and Final Remarks.

Chapter 2

Background & Related Work

Chapter 2: Background & Related Work

2.1. Overview

Firefighter drones, also known as unmanned aerial vehicles UAV, are remote-controlled or autonomous flying platforms equipped with advanced technology. They provide real-time situational awareness, enhance safety, and optimize efficiency during firefighting missions. Key benefits include:

- i) **Information Gathering:** Drones act as scouts, using cameras with thermal imaging technology to aid first responders in rescue efforts.
- ii) **Height and Location Access:** Drones reach heights and locations that firefighters or rescuers cannot access without specialized equipment.
- iii) **Multiple Vantage Points:** They offer multiple views of rooftops, reducing the risk to crews on unstable structures.
- iv) **Indoor Search:** Drones can search for victims in structure fires, even indoors.

2.2. Background

Over the past few years, the adoption of drones within public safety has surged. While police departments dominate drone usage, fire departments have also embraced this technology. Technologies like thermal imaging, real-time data transmission, and autonomous navigation enhance firefighting drone capabilities. Drones assist in identifying victims, monitoring fire intensity, and assessing dangerous situations before human responders enter. Their ability to fly in tight spaces, detect objects, and cover large areas efficiently makes them invaluable tools for firefighting teams.

2.2.1. Historical Context:

The concept of using aerial vehicles for firefighting dates back several decades. Initially, manned helicopters were employed for water drops and reconnaissance during wildfires. However, the limitations of manned aircraft (cost, risk to pilots, and availability) led to the exploration of unmanned alternatives.

2.2.2. Early Adoption:

In the early 2000s, small drones gained popularity for various applications, including firefighting. Fire departments recognized their potential for assessing fire scenes, monitoring hotspots, and aiding search-and-rescue operations.

2.2.3. Technological Advancements:

Advances in drone technology have been pivotal:

- i) **Miniaturization:** Smaller, lightweight drones are easier to deploy quickly.

- ii) **Payload Capacity:** Modern drones can carry payloads like thermal cameras, gas sensors, and even fire extinguishing agents.
- iii) **Autonomous Navigation:** Drones can fly predefined routes or autonomously explore areas.
- iv) **Real-Time Data Transmission:** Live video feeds and telemetry enhance situational awareness.
- v) **Battery Life:** Longer flight times allow extended monitoring.
- vi) **Obstacle Avoidance:** Improved collision avoidance systems to prevent accidents.

These advancements have made drones indispensable tools for firefighting teams.

2.2.4. Use Cases:

Firefighter drones are used in various scenarios:

- i) **Wildfires:** Assessing fire spread, locating hotspots, and monitoring containment efforts.
- ii) **Urban Fires:** Providing real-time visuals to incident commanders.
- iii) **Search and Rescue:** Locating missing persons in challenging terrain.
- iv) **Hazardous Materials Incidents:** Detecting toxic gases or assessing chemical spills.
- v) **Building Inspections:** Checking rooftops, vents, and inaccessible areas.
- vi) **Training:** Simulating emergency scenarios for firefighter training.

2.2.5 Challenges and Future Directions:

Despite their benefits, challenges remain:

- i) **Regulations:** Compliance with aviation rules and airspace restrictions.
- ii) **Interoperability:** Integrating drones into existing emergency response systems.
- iii) **Privacy Concerns:** Balancing public safety with privacy rights.
- iv) **The future holds promise:**
- v) **Swarm Technology:** Coordinated fleets of drones working together.
- vi) **AI and Machine Learning:** Enhancing object detection and decision-making.
- vii) **Beyond Visual Line of Sight (BVLOS):** Expanding operational range.
- viii) **Firefighting Agents:** Developing specialized payloads for fire suppression.

2.3. Related Work

The study of firefighter drones began with the introduction of UAV for firefighting at the 2011 Fire Department Instructors Conference (FDIC) International in Indianapolis. Since then, thousands of drones have been sold to first responder agencies. However, the adoption of drones within firefighting, search and rescue (SAR), and emergency management has significantly increased over the past few years. While police departments dominate drone usage, fire departments have also expanded their use of drones and related technologies. These advancements include real-time visual observation, thermal imaging, recording capabilities, and logistics applications. Drones now reach new heights, fly longer distances, and offer improved object detection and avoidance technologies. The field continues to evolve, with research papers exploring various aspects of firefighter drones and their impact on public safety.

2.3.1. Fire Fighting Drone Using Extinguisher Bomb By International Research Journal Of Engineering And Technology (IRJET) (April 2020): “[Drones | An Open Access Journal from MDPI](#)”

In this paper, the authors explore the use of Unmanned Aerial Vehicles (UAVs) for firefighting. Specifically, they focus on a Hexacopter drone equipped with a gimbal that holds fire extinguisher balls containing powdered CO₂. These drones can be remotely deployed to extinguish fires, reducing risks to human firefighters. The design features lightweight materials, fast reload capabilities, and durability. Additionally, the paper suggests incorporating thermal sensors, GPS tracking, and a thermal infrared camera for fire detection and localization, enhancing the overall firefighting capabilities of these drones.

2.3.1.1. Design and Proposed Features:

- i) The paper introduces a Hexacopter drone designed for firefighting.
- ii) Lightweight materials, fast reload capabilities, and durability are emphasized.
- iii) The drone carries fire extinguisher balls containing powdered CO₂.
- iv) Technological features include thermal sensors, GPS tracking, and a thermal infrared camera.

2.3.1.2. Features:

- i) Hexacopter design with six 680kv motors for stability and heavy lifting.
- ii) Gimbal holds fire extinguisher balls for remote deployment.
- iii) Thermal sensors aid in fire detection.
- iv) GPS tracking ensures precise localization.
- v) Thermal infrared cameras enhance situational awareness.

2.3.1.3. Summary

- i) UAVs can significantly improve firefighting efficiency.
- ii) The proposed features enhance safety and effectiveness.
- iii) Future research should focus on real-world testing and scalability.

2.3.2. Firefighter Drone by Computer Engineering Department, V.E.S.I.T., Mumbai, India: “(PDF) Fire Fighter Drone (researchgate.net)”

The paper discusses an innovative fire-fighting drone designed to enhance the safety and efficiency of fire response operations. It highlights the alarming rate of fire accidents and fatalities, including those of firemen, underscoring the need for a technological solution. The drone, equipped with cameras, sensors, and a flight controller, aims to be the first responder, providing real-time insights into the situation, detecting humans and hazardous materials, and even dropping fire extinguisher balls. The paper details the construction of the UAV, object detection capabilities, panorama area modeling, and toxicity report generation, all of which contribute to a safer and more informed approach to firefighting. Future enhancements include a robotic arm for extinguisher ball deployment and a thermal camera for heat map generation, with potential applications extending beyond firefighting to areas like surveillance and agriculture. The project promises to make fire response more effective while protecting human lives.

2.3.2.1. Design and Proposed Features:

- i) The paper introduces an innovative fire-fighting drone designed to enhance safety and efficiency.
- ii) Equipped with cameras, sensors, and a flight controller, it acts as the first responder.
- iii) Object detection capabilities allow real-time insights into the situation.
- iv) The drone can detect humans and hazardous materials.

2.3.2.2. Features:

- i) Panorama area modeling provides a comprehensive view of the fire scene.
- ii) Toxicity report generation helps assess risks.
- iii) Future enhancements include a robotic arm for precise extinguisher ball deployment.
- iv) A thermal camera for heat map generation is also planned.

2.3.2.3. Summary

- i) The project aims to make fire response more effective while protecting human lives.
- ii) Beyond firefighting, the technology has applications in surveillance and agriculture.

2.3.3. Firefighting Drone Configuration and Scheduling for Wildfire Based on Loss Estimation and Minimization by School of Information Science and Technology, Hangzhou Normal University, China: “Drones | An Open Access Journal from MDPI”

Firefighting Drone Optimization: The paper introduces a mathematical model to estimate wildfire spread and economic losses, aiming to optimize firefighting drone scheduling. It emphasizes the importance of drones in improving response speed and safety in firefighting operations.

Metaheuristic Optimization: The study compares various metaheuristic optimization algorithms, highlighting the superiority of water wave optimization (WWO) for scheduling drones in simulated wildfire scenarios in Hangzhou, China.

Practical Application: The proposed model and optimization method can pre-define comprehensive plans for drone deployment, enhancing emergency response efficiency and reducing potential losses in real-world wildfire situations.

Innovative Contribution: The paper’s main contribution lies in its simultaneous consideration of fire spread prediction, drone scheduling, and support staff configuration, which is novel in the field of wildfire management.

2.3.3.1. Design/Proposal:

- i) **Mathematical Model:** A mathematical model is presented to estimate wildfire spread and economic losses, determining the minimum number of firefighting drones needed.
- ii) **Drone Scheduling:** Proposes a method for scheduling drones using metaheuristic optimization to minimize expected loss in wildfire scenarios.

2.3.3.2. Features:

- i) **Metaheuristic Optimization:** Utilizes water wave optimization (WWO) among other algorithms for effective drone scheduling.

- ii) **Comprehensive Planning:** Enables pre-defined planning for drone and support staff configuration in response to various wildfire occurrences.

2.3.3.3. Conclusion:

- i) **Efficiency Improvement:** The proposed method significantly improves emergency response efficiency and reduces potential losses.
- ii) **Validation:** The model and optimization method are validated through computational experiments, demonstrating effectiveness in real-world instances.

2.3.4. Enhancing Firefighting Efficiency: Manual ESC-equipped Quadcopters with KK 2.1.5 Flight Controller and Fire Extinguisher Balls (Deemed University), Pune, India:

https://www.techrxiv.org/users/691038/articles/681901/master/file/data/PBL3_final_paper_2may2023/PBL3_final_paper_2may2023.pdf

The paper discusses the integration of manual ESC-equipped quadcopters with KK 2.1.5 flight controllers and fire extinguisher balls to enhance firefighting efficiency. It explores the use of drones in various firefighting scenarios, such as detecting and monitoring fires, providing real-time data to firefighters, and assisting in search and rescue operations. The paper also addresses the challenges of drone use in firefighting, including regulatory, technical, and ethical issues, and suggests continued research and development to fully realize the potential of drones in modern firefighting. Key aspects include the design and testing of a drone equipped with a ball dispenser system operated by servo motors and an RC controller, demonstrating the drone's ability to accurately and efficiently extinguish fires.

2.3.4.1. Design/Proposed System:

- i) The project involves designing a drone equipped with firefighting balls and a box operated by servo motors and an RC controller.
- ii) It includes a quadcopter frame, motors, electronic speed controller, flight controller, camera, GPS module, and a ball dispenser system.

2.3.4.2. Features:

- i) The drone can detect and extinguish fires using precision-dropped firefighting balls.
- ii) It has a custom mounting system for the ball dispenser and is controlled remotely via an RC controller designed with various components.

2.3.4.3. Conclusion:

- i) The hardware project demonstrates potential to revolutionize firefighting operations with its ability to accurately detect and extinguish fires.
- ii) Further research and development could lead to more advanced drones capable of handling larger fires and operating in challenging environments.

2.3.5. Use of drones for firefighting operations (Aalborg University, Esbjerg, Denmark): “Microsoft Word - DRTHESES almost done (aau.dk)”

The Master Thesis Report by Vlad Tiberiu Radu from Aalborg University, Denmark, explores the use of drones in firefighting operations. It delves into how drones, once primarily military tools, have evolved for commercial and public safety uses, particularly in firefighting. The research includes qualitative semi-structured interviews with decision-makers in Falck Fire Services to assess the current implementation stage, benefits, usage, limitations, and future prospects of drone technology in firefighting. The study also examines legislative developments within the EU and identifies drones capable of meeting firefighting requirements. The report concludes with a schematic analysis of drone usage in emergency management and a Bow-Tie model that identifies risks and barriers associated with drone crashes. The thesis encourages further research on safely integrating drones into firefighting and other applications.

2.3.5.1. Design/Proposed:

The thesis proposes the integration of drones into Falck Fire Services, analyzing how drones can enhance firefighting operations by providing real-time information, identifying fire hydrants, and aiding in strategic decision-making.

2.3.5.2. Features:

Drones offer various features such as live footage transmission, thermal and optical cameras for better visibility, and the ability to operate in challenging environments. They can support all phases of emergency management: mitigation, preparedness, response, and recovery.

2.3.5.3. Conclusion:

The report concludes that drones have significant potential to improve firefighting activities by reducing response times, increasing safety for people and assets, and enhancing operational effectiveness². It also identifies the need for industry-standard regulations for drone use in emergency services.

2.3.5.4. Future Scope:

The thesis encourages future research on safely introducing drones into firefighting activities, considering technological advancements, legislative developments, and the creation of industry standards for emergency responders' drone use. It suggests further exploration of drones' capabilities in various emergency management stages.

2.4. Summary:

- **Overview:** Unmanned aerial vehicles (UAVs) equipped with advanced technology for real-time situational awareness and efficiency in firefighting missions.
- **Benefits:** Include information gathering, access to challenging locations, multiple vantage points, indoor search, and extended flight times.
- **Technological Advancements:**
 - **Historical Context:** Shift from manned helicopters to drones due to cost, risk, and availability.
 - **Technological Progress:** Miniaturization, payload capacity, autonomous navigation, real-time data transmission, battery life, and obstacle avoidance.
- **Use Cases & Challenges:**
 - **Applications:** Wildfires, urban fires, search and rescue, hazardous materials incidents, building inspections, and training.
 - **Challenges:** Regulations, interoperability, and privacy concerns.
- **Related Work:**
 - **Research Papers:** Studies on drones' use in firefighting, including extinguisher bombs, object detection, and optimization models for drone scheduling.
- **Future Directions:**
 - **Innovations:** Swarm technology, AI and machine learning, BVLOS operations, and specialized payloads for fire suppression.

CHAPTER 3

Simulation Using ROS

CHAPTER 3: Simulation Using ROS

3.1. Overview:

Simulation is the process of using a digital or mathematical model to represent a real-world system or phenomenon in order to study its behavior under different conditions. Simulations are widely used in various fields such as engineering, science, economics, and medical training. It contains a lot of plugins that is used for making our world and the characteristics of the frame of robot that need to be design and sensors and actuators related to object and communication between robot component and how to interact with the environment.

3.2. The main components of simulation

3.2.1 Gazebo

Gazebo is a powerful open-source robotics simulation software that provides a realistic and dynamic environment for testing and developing robotic systems. It offers a physics engine, sensor simulations, and visualization capabilities, allowing users to simulate and analyze the behavior of robots in various scenarios. Gazebo is widely used in research, education, and industry to design, prototype, and evaluate robotic systems before deploying them in the real world. A Physical Gazebo In the world of parks, gardens, and backyards, a gazebo is a freestanding structure with a roof and open sides. It's a decorative element that provides shade, shelter from the elements, and a pleasant spot to relax or entertain guests. Gazebo Simulator In the realm of robotics research and development, Gazebo refers to a powerful 3D robot simulator. Unlike a game engine, Gazebo prioritizes high-fidelity physics simulation to create realistic environments for testing robots. It offers features like Multiple physics engines Simulate robot behavior under different physical conditions. Variety of sensors Simulate real-world sensors like cameras, LiDAR, and accelerometers. User and programmatic interfaces: Interact with the simulation visually or through code.

Install gazebo for ubuntu 20.04

- i) **Setup system to accept software from Index of / (osrfoundation.org):**
`sudo sh -c 'echo "deb http://packages.osrfoundation.org/gazebo/ubuntu-stable `lsb_release -cs` main" > /etc/apt/sources.list.d/gazebo-stable.list'`
- ii) **Setup keys:** `wget http://packages.osrfoundation.org/gazebo.key -O - | sudo apt-key add -`
- iii) **Install gazebo:** `sudo apt-get install gazebo11 libgazebo11-dev`
- iv) **Gazebo plugin for apm2.8:** `cd ~ git clone https://github.com/khancyr/ardupilot_gazebo.git`

`cd ardupilot_gazebo`

build our plugin:

```
mkdir build
cd build
cmake.
make -j4
sudo make install
```

3.2.2Ros

The robotics operating system that is used to connect sensors over messages and topics to communicate with the environment for our drone First, we need to install ros (noetic) that require linux (ubuntu 20.04) and install using series of command and frameworks:

i) 1.Setup your sources. list with command:

- `sudo sh -c 'echo "deb http://packages.ros.org/ros/ubuntu $(lsb_release -sc) main" > /etc/apt/sources.list.d/ros-latest.list'`

ii) install ROS (noetic):

```
sudo apt install ros-noetic-desktop-full
```

iii) we need to setup our environment:

```
source /opt/ros/noetic/setup.bash
```

iv) reload bashrc file in ubuntu

```
echo "source /opt/ros/noetic/setup.bash" >> ~/. bashrc
source ~/. bashrc
```

v) Dependencies for building packages

```
sudo apt install python3-rosdep python3-rosinstall python3-rosinstall-generator python3-wstool build-essential
```

Second, we need to create our work space in which we will make our packages and building for our simulation. Packages that have been used are as follows:

- 1.IQ-SIM
- 2.IQ-GNC
- 3.Darknet-Ros

We need to setup our workspace by some few of steps:

Sudo apt-get install python-wstool python-rosinstall-generator python-catkin-tools

And then we will create a folder called drone work space and inside it we created a folder called source and the packages that were identified above will be uploaded to the source

Initialize our work space:

```
mkdir -p ~/drone_ws/src
```

```
cd ~/drone_ws
```

```
catkin init
```

Then we need to install some additional packages that will help us in identifying flight controller and the communication between the flight controller and raspberry pi controller for making a high processing technique like object detection.

The first additional package is mavros that is a package provides communication driver for various autopilots with MAVLink communication protocol. Additional it provides UDP MAVLink bridge for ground control stations (e.g., QGroundControl).

Main node can be extended by plugins (see [pluginlib](#)). See also [mavros_extras](#) package and we install it and building it in our drone work space using:

```
cd ~/drone_ws
```

```
wstool init ~/drone_ws/src
```

```
rosinstall_generator --upstream mavros | tee /tmp/mavros.rosinstall
```

```
rosinstall_generator mavlink | tee -a /tmp/mavros.rosinstall
```

```
wstool merge -t src /tmp/mavros.rosinstall
```

```
wstool update -t src
```

```
rosdep install --from-paths src --ignore-src --rosdistro `echo $ROS_DISTRO` -y
```

catkin build

```
echo "source ~/catkin_ws/devel/setup.bash" >> ~/. bashrc
```

We can read areal telemetry from mavros topics and these is done by running roscore and see ros topic list that will be shown as:

```
/Clock
```

```
/gazebo/link_states
```

```
/gazebo/model_states
```

```
/gazebo/parameter_descriptions
```

```
/gazebo/parameter_updates
```

```
/gazebo/set_link_state
```

```
/gazebo/set_model_state
```

```
/rosout
```

```
/rosout_agg
/webcam/camera_info
/webcam/image_raw
/webcam/image_raw/compressed
/webcam/image_raw/compressed/parameter_descriptions
/webcam/image_raw/compressed/parameter_updates
/webcam/image_raw/compressedDepth
/webcam/image_raw/compressedDepth/parameter_descriptions
/webcam/image_raw/compressedDepth/parameter_updates
/webcam/image_raw/theora
/webcam/image_raw/theora/parameter_descriptions
/webcam/image_raw/theora/parameter_updates
/webcam/parameter_descriptions
/webcam/parameter_updates
```

Using MAVROS to get telemetry data from the FCU now the topic `/gazebo/model_states` is the true model position in the simulator. This isn't something we can use in real life. In real life, we have to use the estimate of the drone's position which is formulated from a combination of its sensors. This position is transmitted using a communication protocol called mavlink.

These messages are stripped down and are optimized for radio transmission MAVROS is a middle man which translates the MAVlink messages into ROS messages, which are easy to use and common between different robot systems.

To start mavros run following commands
`roslaunch iq_sim apm.launch`
and then run `rostopic` again and will see a different messages shown

```
/clock
/diagnostics
/gazebo/link_states
/gazebo/model_states
/gazebo/parameter_descriptions
/gazebo/parameter_updates
/gazebo/set_link_state
/gazebo/set_model_state
/mavlink/from
/mavlink/to
/mavros/adsb/send
/mavros/adsb/vehicle
/mavros/battery
/mavros/cam_imu_sync/cam_imu_stamp
/mavros/companion_process/status
/mavros/distance_sensor/rangefinder_pub
/mavros/distance_sensor/rangefinder_sub
/mavros/extended_state
/mavros/fake_gps/mocap/tf
/mavros/global_position/compass_hdg
/mavros/global_position/global
/mavros/global_position/gp_lp_offset
/mavros/global_position/gp_origin
```

/mavros/global_position/home
/mavros/global_position/local
/mavros/global_position/raw/fix
/mavros/global_position/raw/gps_vel
/mavros/global_position/rel_alt
/mavros/global_position/set_gp_origin
/mavros/gps_rtk/send_rtcn
/mavros/home_position/home
/mavros/home_position/set
/mavros/imu/data
/mavros/imu/data_raw
/mavros/imu/diff_pressure
/mavros/imu/mag
/mavros/imu/static_pressure
/mavros/imu/temperature_baro
/mavros/imu/temperature_imu
/mavros/landing_target/lt_marker
/mavros/landing_target/pose
/mavros/landing_target/pose_in
/mavros/local_position/accel
/mavros/local_position/odom
/mavros/local_position/pose
/mavros/local_position/pose_cov
/mavros/local_position/velocity_body
/mavros/local_position/velocity_body_cov
/mavros/local_position/velocity_local
/mavros/log_transfer/raw/log_data
/mavros/log_transfer/raw/log_entry
/mavros/manual_control/control
/mavros/manual_control/send
/mavros/mission/reached
/mavros/mission/waypoints
/mavros/mocap/pose
/mavros/obstacle/send
/mavros/odometry/in
/mavros/odometry/out
/mavros/param/param_value
/mavros/radio_status
/mavros/rangefinder/rangefinder
/mavros/rc/in
/mavros/rc/out
/mavros/rc/override
/mavros/setpoint_accel/accel
/mavros/setpoint_attitude/cmd_vel
/mavros/setpoint_attitude/thrust
/mavros/setpoint_position/global
/mavros/setpoint_position/local
/mavros/setpoint_raw/attitude
/mavros/setpoint_raw/global
/mavros/setpoint_raw/local

```
/mavros/setpoint_raw/target_attitude
/mavros/setpoint_raw/target_global
/mavros/setpoint_raw/target_local
/mavros/setpoint_velocity/cmd_vel
/mavros/setpoint_velocity/cmd_vel_unstamped
/mavros/state
/mavros/statustext/recv
/mavros/statustext/send
/mavros/time_reference
/mavros/timesync_status
/mavros/trajectory/desired
/mavros/trajectory/generated
/mavros/trajectory/path
/mavros/vfr_hud
/mavros/vision_pose/pose
/mavros/vision_pose/pose_cov
/mavros/wind_estimation
/rosout
/rosout_agg
/tf
/tf_static
/webcam/camera_info
/webcam/image_raw
/webcam/image_raw/compressed
/webcam/image_raw/compressed/parameter_descriptions
/webcam/image_raw/compressed/parameter_updates
/webcam/image_raw/compressedDepth
/webcam/image_raw/compressedDepth/parameter_descriptions
/webcam/image_raw/compressedDepth/parameter_updates
/webcam/image_raw/theora
/webcam/image_raw/theora/parameter_descriptions
/webcam/image_raw/theora/parameter_updates
/webcam/parameter_descriptions
/webcam/parameter_updates
```

Now we can see the drone's position in its local frame by running rostopic echo /mavros/global_position/local

In the following tutorials we will be accessing the data on these topics in our C++ programs. To see the type of message being published run

```
rostopic list -v /mavros/global_position/local
```

We see that the topic is publishing the message in the form of mav_msgs/Odometry

to see the structure of the message you can run the following command. rosmmsg show mav_msgs/Odometry

This will be usefull when writing publishers and subscribers in the future

second additional package is mavlink it we need and to install to install mavproxy configuration protocal and install it using :

```
sudo pip install future pymavlink MAVProxy
```

And we need to edit our bashrc file to ensure that mavproxy add to the system:

```
gedit ~/. bashrc
```

```
export PATH=$PATH: $HOME/ardupilot/Tools/autotest
```

```
export PATH=/usr/lib/ccache: $PATH
```

```
. ~/. bashrc
```

Now, we can install mavlink for controlling our drone using:

```
git clone https://github.com/mavlink/mavlink.git --recursive
```

```
python3 -m pip install -r pymavlink/requirements.txt
```

We install everything to create our package for our drone and the world in gazebo let's do it the first package we will create is iq_sim

1.IQ-Sim

We needed to build our drone and the other world, and then we went in to make our model, which is a drone quadcopter, is a type of helicopter or multicopter that has four rotors. the frame for the drone, which we built by model.sdf file:

```

<?xml version='1.0'?>
<sdf version="1.6" >
  <model name="iris_demo">
    <include>
      <uri>model://iris_base</uri>
    </include>

    <!-- plugins -->
    <plugin name="rotor_0_blade_1" filename="libLiftDragPlugin.so">
      <a0>0.3</a0>
      <alpha_stall>1.4</alpha_stall>
      <cla>4.2500</cla>
      <cda>0.10</cda>
      <cma>0.00</cma>
      <cla_stall>-0.025</cla_stall>
      <cda_stall>0.0</cda_stall>
      <cma_stall>0.0</cma_stall>
      <area>0.002</area>
      <air_density>1.2041</air_density>
      <cp>0.084 0 0</cp>
      <forward>0 1 0</forward>
      <upward>0 0 1</upward>
      <link_name>iris::rotor_0</link_name>
    </plugin>
    <plugin name="rotor_0_blade_2" filename="libLiftDragPlugin.so">
      <a0>0.3</a0>
      <alpha_stall>1.4</alpha_stall>
      <cla>4.2500</cla>
      <cda>0.10</cda>
      <cma>0.00</cma>
      <cla_stall>-0.025</cla_stall>
      <cda_stall>0.0</cda_stall>
      <cma_stall>0.0</cma_stall>
      <area>0.002</area>
      <air_density>1.2041</air_density>
      <cp>-0.084 0 0</cp>
      <forward>0 -1 0</forward>
      <upward>0 0 1</upward>
      <link_name>iris::rotor_0</link_name>
    </plugin>

    <plugin name="rotor_1_blade_1" filename="libLiftDragPlugin.so">
      <a0>0.3</a0>
      <alpha_stall>1.4</alpha_stall>
      <cla>4.2500</cla>
      <cda>0.10</cda>
      <cma>0.00</cma>
      <cla_stall>-0.025</cla_stall>
      <cda_stall>0.0</cda_stall>
      <cma_stall>0.0</cma_stall>
      <area>0.002</area>
      <air_density>1.2041</air_density>
      <cp>0.084 0 0</cp>

```

```

        <i_max>0</i_max>
        <i_min>0</i_min>
        <cmd_max>2.5</cmd_max>
        <cmd_min>-2.5</cmd_min>
        <jointName>iris::rotor_0_joint</jointName>
        <multiplier>838</multiplier>
        <controlVelocitySlowdownSim>1</controlVelocitySlowdownSim>
    </control>
    <control channel="1">
        <type>VELOCITY</type>
        <offset>0</offset>
        <p_gain>0.20</p_gain>
        <i_gain>0</i_gain>
        <d_gain>0</d_gain>
        <i_max>0</i_max>
        <i_min>0</i_min>
        <cmd_max>2.5</cmd_max>
        <cmd_min>-2.5</cmd_min>
        <jointName>iris::rotor_1_joint</jointName>
        <multiplier>838</multiplier>
        <controlVelocitySlowdownSim>1</controlVelocitySlowdownSim>
    </control>
    <control channel="2">
        <type>VELOCITY</type>
        <offset>0</offset>
        <p_gain>0.20</p_gain>
        <i_gain>0</i_gain>
        <d_gain>0</d_gain>
        <i_max>0</i_max>
        <i_min>0</i_min>
        <cmd_max>2.5</cmd_max>
        <cmd_min>-2.5</cmd_min>
        <jointName>iris::rotor_2_joint</jointName>
        <multiplier>-838</multiplier>
        <controlVelocitySlowdownSim>1</controlVelocitySlowdownSim>
    </control>
    <control channel="3">
        <type>VELOCITY</type>
        <offset>0</offset>
        <p_gain>0.20</p_gain>
        <i_gain>0</i_gain>
        <d_gain>0</d_gain>
        <i_max>0</i_max>
        <i_min>0</i_min>
        <cmd_max>2.5</cmd_max>
        <cmd_min>-2.5</cmd_min>
        <jointName>iris::rotor_3_joint</jointName>
        <multiplier>-838</multiplier>
        <controlVelocitySlowdownSim>1</controlVelocitySlowdownSim>
    </control>
</plugin>

</model>
</sdf>

```

We needed to make our own world and we would create it in drone. world file that contains are as follows

- 1.gazebo world
- 2.Camera sensor
- 3.Lidear sensor

We need to make our world and insert our drone on it and that make by:

```
:?xml version="1.0" ?>
:sdf version="1.6">
  <world name="default">
    <physics type="ode">
      <ode>
        <solver>
          <type>quick</type>
          <iters>100</iters>
          <sor>1.0</sor>
        </solver>
        <constraints>
          <cfm>0.0</cfm>
          <erp>0.9</erp>
          <contact_max_correcting_vel>0.1</contact_max_correcting_vel>
          <contact_surface_layer>0.0</contact_surface_layer>
        </constraints>
      </ode>
      <real_time_update_rate>-1</real_time_update_rate>
      <!-- <max_step_size>0.0020</max_step_size> -->
    </physics>

    <include>
      <uri>model://sun</uri>
    </include>

    <include>
      <uri>model://ground_plane</uri>
    </include>

    <model name="iris">
      <include>
        <uri>model://iris_with_standoffs_demo</uri>
      </include>
```

First sensor is camera we need to identify

Camera's (link, joint)

```

<link name='camera'>
  <pose>0 -0.01 0.070 1.57 0 1.57</pose>
  <inertial>
    <pose>0 0 0 0 0 0</pose>
    <mass>0.1</mass>
    <inertia>
      <ixx>0.001</ixx>
      <ixy>0</ixy>
      <ixz>0</ixz>
      <iyy>0.001</iyy>
      <iyz>0</iyz>
      <izz>0.001</izz>
    </inertia>
  </inertial>
  <visual name='visual'>
    <pose>0 0 0 0 0 0</pose>
    <geometry>
      <cylinder>
        <radius>0.025</radius>
        <length>0.025</length>
      </cylinder>
    </geometry>
    <material>
      <script>
        <uri>file://media/materials/scripts/gazebo.material</uri>
        <name>Gazebo/Grey</name>
      </script>
    </material>
  </visual>

  <sensor name="camera" type="camera">
    <pose>0 0 0 -1.57 -1.57 0</pose>
    <camera>
      <horizontal_fov>1.0472</horizontal_fov>
      <image>
        <width>640</width>
        <height>480</height>
      </image>
      <clip>
        <near>0.05</near>
        <far>1000</far>
      </clip>
    </camera>
    <always_on>1</always_on>
    <update_rate>10</update_rate>
    <visualize>true</visualize>

    <!-- <plugin name="irlock" filename="libArduCopterIRLockPlugin.so">
      <fiducial>irlock_beacon_01</fiducial>
    </plugin> -->
    <plugin name="camera_controller" filename="libgazebo_ros_camera.so">
      <alwaysOn>true</alwaysOn>
      <updateRate>0.0</updateRate>
      <cameraName>webcam</cameraName>
      <imageTopicName>image_raw</imageTopicName>

```

```

<!-- attach camera -->
<joint type="revolute" name="base_camera_joint">
  <pose>0 0 0.0 0 0 0</pose>
  <parent>iris::iris_demo::gimbal_small_2d::tilt_link</parent>
  <child>camera</child>
  <axis>
    <limit>
      <lower>0</lower>
      <upper>0</upper>
    </limit>
    <xyz>0 0 1</xyz>
    <use_parent_model_frame>true</use_parent_model_frame>
  </axis>
</joint>

```

Second sensor is lidar that is used for obstacle avoidance and like camera we need to identify lidar (link, joint)

```

<!--add lidar-->
<link name="hokuyo_link">
  <pose>0 0 0 0 0 0</pose>
  <collision name="collision">
    <pose>0 0 0.3 0 0 0</pose>
    <geometry>
      <box>
        <size>0.1 0.1 0.1</size>
      </box>
    </geometry>
  </collision>
  <visual name="visual">
    <pose>0 0 0.27 0 0 0</pose>
    <geometry>
      <mesh>
        <uri>model://hokuyo/meshes/hokuyo.dae</uri>
      </mesh>
    </geometry>
  </visual>
  <inertial>
    <mass>0.016</mass>
    <inertia>
      <ixx>0.0001</ixx>
      <ixy>0</ixy>
      <ixz>0</ixz>
      <iyy>0.0001</iyy>
      <iyz>0</iyz>
      <izz>0.0001</izz>
      <!-- low inertia necessary to avoid not disturb the drone -->
    </inertia>
  </inertial>

  <sensor type="ray" name="laser">
    <pose>0 0 0.3 0 0 1.57</pose>

```

```

<sensor type="ray" name="laser">
  <pose>0 0 0.3 0 0 1.57</pose>
  <visualize>true</visualize>
  <update_rate>10</update_rate>
  <ray>
    <scan>
      <horizontal>
        <samples>1024</samples>
        <resolution>1</resolution>
        <min_angle>-3.141593</min_angle>
        <max_angle>3.141593</max_angle>
      </horizontal>
    </scan>
    <range>
      <min>0.1</min>
      <max>30</max>
      <resolution>0.1</resolution>
    </range>
    <!-- <noise>
      <type>Gaussian</type>
      <mean>0.0</mean>
      <stddev>0.01</stddev>
    </noise> -->
  </ray>
  <plugin name="hokuyo_node" filename="libgazebo_ros_laser.so">
    <robotNamespace></robotNamespace>
    <topicName>/spur/laser/scan</topicName>
    <frameName>/hokuyo_sensor_link</frameName>
  </plugin>
</sensor>
</link>

```

```

<joint name="hokuyo_joint" type="fixed">
  <pose>0 0 0 0 0 0</pose>
  <parent>iris::iris_demo::iris::base_link</parent>
  <child>hokuyo_link</child>
</joint>

```

```

</model>

```

We need to create drone.launch file to launch the drone and the world in gazebo that's helping us to launch our drone with sensors in the gazebo

```
<launch>
<!-- We resume the logic in empty_world.launch, changing only the name of the world to be launched -->
<include file="$(find gazebo_ros)/launch/empty_world.launch">
  <arg name="world_name" value="$(find iq_sim)/worlds/droneOnly.world"/>
  <!-- more default parameters can be changed here -->
</include>
</launch>
```

Finally, we need to build our work space to ensure everything is running well and reload the bashrc file in ubuntu to ensure the devel/setup.bash file is saved on it .

And now we need to open the terminal and launch the gazebo world

```
roslaunch iq_sim droneOnly.launch
```

now gazebo launch and the world appear like that

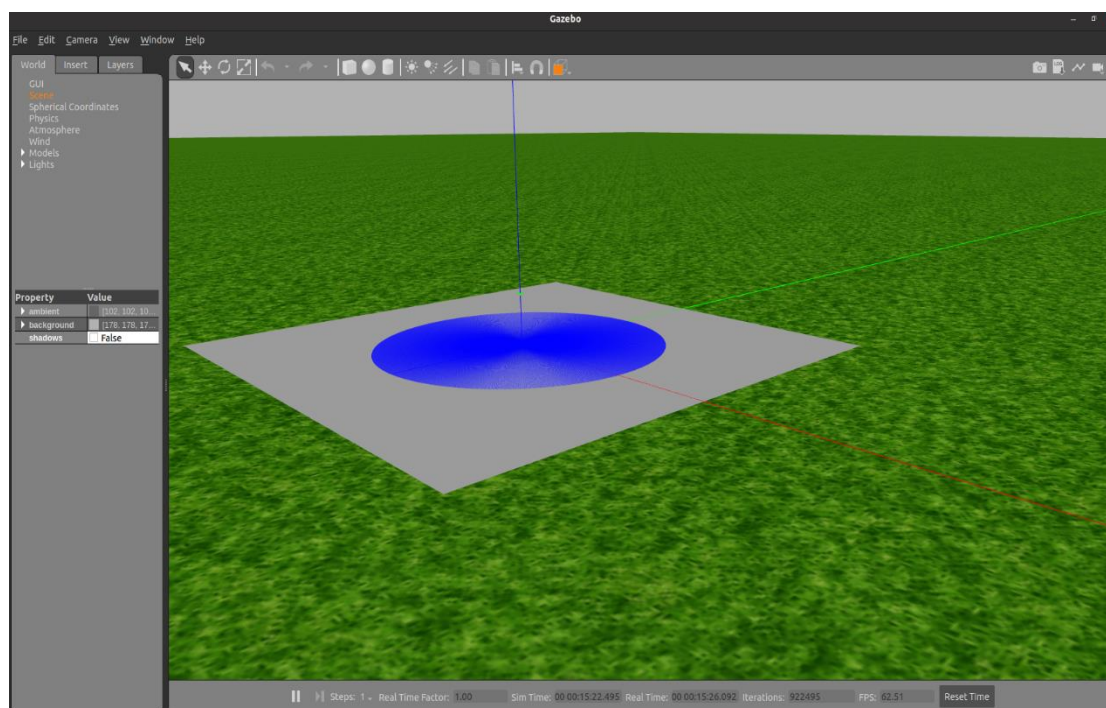


Figure 1: workspace

Now, we will create the second package for making our drone to perform some activity like moving around ground plane to detect object and the obstacle avoidance

2.IQ_gnc

First, we will create a python code for obstacle avoidance:

```
1 import rospy
2 # Import LaserScan message from package sensor_msgs.
3 from sensor_msgs.msg import LaserScan
4 # Import the API.
5 from iq_gnc.py_gnc_functions import *
6 # Import the needed math functions.
7 from math import cos, sin, pow, radians, sqrt
8 # Importing Point message from package geometry_msgs.
9 from geometry_msgs.msg import Point
10
11 # Create an object for the API and making it a global variable.
12 drone = gnc_api()
13
14 def laser_cb(msg):
15     # Callback function of the subscriber.
16     cr_scan = LaserScan()
17     cr_scan = msg
18     avoid_x = 0.0
19     avoid_y = 0.0
20     avoid = False
21
22     for i in range(1, len(cr_scan.ranges)):
23         d0 = 3.0
24         k = 0.5
25         if cr_scan.ranges[i] < d0 and cr_scan.ranges[i] > 0.35:
26             avoid = True
27             x = cos(cr_scan.angle_increment * i)
28             y = sin(cr_scan.angle_increment * i)
29             u = (-0.5 * k * pow(((1/cr_scan.ranges[i]) - (1/d0)), 2.0))
30
31             avoid_x += (x*u)
32             avoid_y += (y*u)
33
34     # Getting the current_heading of the drone and converting it to radian
35     cr_heading = radians(drone.get_current_heading())
36     avoid_x = (avoid_x * cos(cr_heading)) - (avoid_y * sin(cr_heading))
37     avoid_y = (avoid_x * sin(cr_heading)) + (avoid_y * cos(cr_heading))
38
39     if avoid:
40         dist = sqrt(pow(avoid_x, 2) + pow(avoid_y, 2))
41
42         if dist > 3:
43             avoid_x = (3 * (avoid_x/dist))
44             avoid_y = (3 * (avoid_y/dist))
45
46     cur_pose = Point()
47     # Getting the current location from the drone.
48     cur_pose = drone.get_current_location()
49     # Sending the goal.
50     drone.set_destination(avoid_x + cur_pose.x,
```

```

        avoid_x += (x*u)
        avoid_y += (y*u)

# Getting the current_heading of the drone and converting it to radians.
cr_heading = radians(drone.get_current_heading())
avoid_x = (avoid_x * cos(cr_heading)) - (avoid_y * sin(cr_heading))
avoid_y = (avoid_x * sin(cr_heading)) + (avoid_y * cos(cr_heading))

if avoid:
    dist = sqrt(pow(avoid_x, 2) + pow(avoid_y, 2))

    if dist > 3:
        avoid_x = (3 * (avoid_x/dist))
        avoid_y = (3 * (avoid_y/dist))

    cur_pose = Point()
    # Getting the current location from the drone.
    cur_pose = drone.get_current_location()
    # Sending the goal.
    drone.set_destination(avoid_x + cur_pose.x,
                          avoid_y + cur_pose.y,
                          2, 0)

def main():
    # Initializing the ROS node.
    rospy.init_node("obs_avoider", anonymous=True)
    # Creating a subscriber for the topic '/spur/laser/scan'.
    rospy.Subscriber(name="/spur/laser/scan",
                     data_class=LaserScan,
                     queue_size=1,
                     callback=laser_cb)

    # Wait for FCU connection.
    drone.wait4connect()
    # Wait for the mode to be switched.
    drone.wait4start()
    # Create local reference frame.
    drone.initialize_local_frame()
    # Request takeoff with an altitude of 2m.
    drone.takeoff(2)

    # Used to keep the node running.
    rospy.spin()

if __name__ == '__main__':
    try:
        main()
    except KeyboardInterrupt:

```

And, we can run this file by:

Rosrun iq_gnc obs_avoid.py

We can now make a script for moving drone around ground plane using square.cpp file as shown below We should move our drone using three-axis (x, y, z) that identify in real (roll, pitch, yaw)

Roll: Rotation around front-to-back axis

Pitch: Rotation around side-to-side axis

Yaw: Rotation around vertical axis.

We also should identify some function such as:

Wait4connect function: that return 0 if have a copter connect

Wait4start function: that return 0 if

The copter run the script

```

#include <gnc_functions.hpp>
//include API

int main(int argc, char** argv)
{
    //initialize ros
    ros::init(argc, argv, "gnc_node");
    ros::NodeHandle gnc_node("~");

    //initialize control publisher/subscribers
    init_publisher_subscriber(gnc_node);

    // wait for FCU connection
    wait4connect();

    //wait for used to switch to mode GUIDED
    wait4start();

    //create local reference frame
    initialize_local_frame();

    //request takeoff
    takeoff(3);

    //specify some waypoints
    std::vector<gnc_api_waypoint> waypointList;
    gnc_api_waypoint nextWayPoint;
    nextWayPoint.x = 0;
    nextWayPoint.y = 0;
    nextWayPoint.z = 3;
    nextWayPoint.psi = 0;
    waypointList.push_back(nextWayPoint);
    nextWayPoint.x = 5;
    nextWayPoint.y = 0;
    nextWayPoint.z = 3;
    nextWayPoint.psi = -90;
    waypointList.push_back(nextWayPoint);
    nextWayPoint.x = 5;
    nextWayPoint.y = 5;
    nextWayPoint.z = 3;
    nextWayPoint.psi = 0;
    waypointList.push_back(nextWayPoint);
    nextWayPoint.x = 0;
    nextWayPoint.y = 5;
    nextWayPoint.z = 3;
    nextWayPoint.psi = 90;
    waypointList.push_back(nextWayPoint);
    nextWayPoint.x = 0;
    nextWayPoint.y = 0;
    nextWayPoint.z = 3;
    nextWayPoint.psi = 180;
    waypointList.push_back(nextWayPoint);

```

```

nextWayPoint.z = 3;
nextWayPoint.psi = 0;
waypointList.push_back(nextWayPoint);
nextWayPoint.x = 5;
nextWayPoint.y = 0;
nextWayPoint.z = 3;
nextWayPoint.psi = -90;
waypointList.push_back(nextWayPoint);
nextWayPoint.x = 5;
nextWayPoint.y = 5;
nextWayPoint.z = 3;
nextWayPoint.psi = 0;
waypointList.push_back(nextWayPoint);
nextWayPoint.x = 0;
nextWayPoint.y = 5;
nextWayPoint.z = 3;
nextWayPoint.psi = 90;
waypointList.push_back(nextWayPoint);
nextWayPoint.x = 0;
nextWayPoint.y = 0;
nextWayPoint.z = 3;
nextWayPoint.psi = 180;
waypointList.push_back(nextWayPoint);
nextWayPoint.x = 0;
nextWayPoint.y = 0;
nextWayPoint.z = 3;
nextWayPoint.psi = 0;
waypointList.push_back(nextWayPoint);

//specify control loop rate. We recommend a low frequency to not over load the FCU with messages. Too many messages will cause the drone to be sluggish
ros::Rate rate(2.0);
int counter = 0;
while(ros::ok())
{
    ros::spinOnce();
    rate.sleep();
    if(check_waypoint_reached(.3) == 1)
    {
        if (counter < waypointList.size())
        {
            set_destination(waypointList[counter].x, waypointList[counter].y, waypointList[counter].z, waypointList[counter].psi);
            counter++;
        }
        else{
            //land after all waypoints are reached
            land();
        }
    }
}
return 0;

```

We can now run this script using the following code:

Ros run iq_gnc square.cpp

Now the last package to talk about is dark_net_ros this package allows to make object detection using yolo pretrained models but we need to install some dependencies like Cuda is a library that allows programs to take advantage of your GPU as a computing resource. YOLO will run without Cuda, but the algorithm is up to 500 x quicker with Cuda. To install Cuda, run to install cuda.

sudo apt install nvidia-cuda-toolkit

Then we clone the package in the workspace in src folder:

cd ~/drone_ws/src

git clone --recursive https://github.com/leggedrobotics/darknet_ros.git.

Now build the package: catkin build -DCMAKE_BUILD_TYPE=Release

we need to configure yolo in the file ros. yaml specifies ros parameters You can find this file under darknet_ros/darknet_ros/config.

You will need to change the image topic from /camera/rgb/image_raw to /webcam/image_raw.

The file darknet_ros.launch will launch the darknet/yolo ros node. You can find this file under darknet_ros/darknet_ros/launch in this file you can choose which version of yolo you would like to run by changing <arg name="network_param_file" default="\$(find darknet_ros) /config/yolov2-tiny.yaml"/>.

the options are as follows:

yolov1: Not recommended. this model is old

yolov2: more accurate, and faster.

yolov3: about as fast as v2, but more accurate. Yolo v3 has a high GPU ram requirement to train and run. If your graphics card does not have enough ram, use yolo v2

tiny-yolo: Very fast yolo model. Would recommend for application where speed is most important. Works very well on Nvidia Jetson

And finally, the drone simulation will appear in gazebo like that:

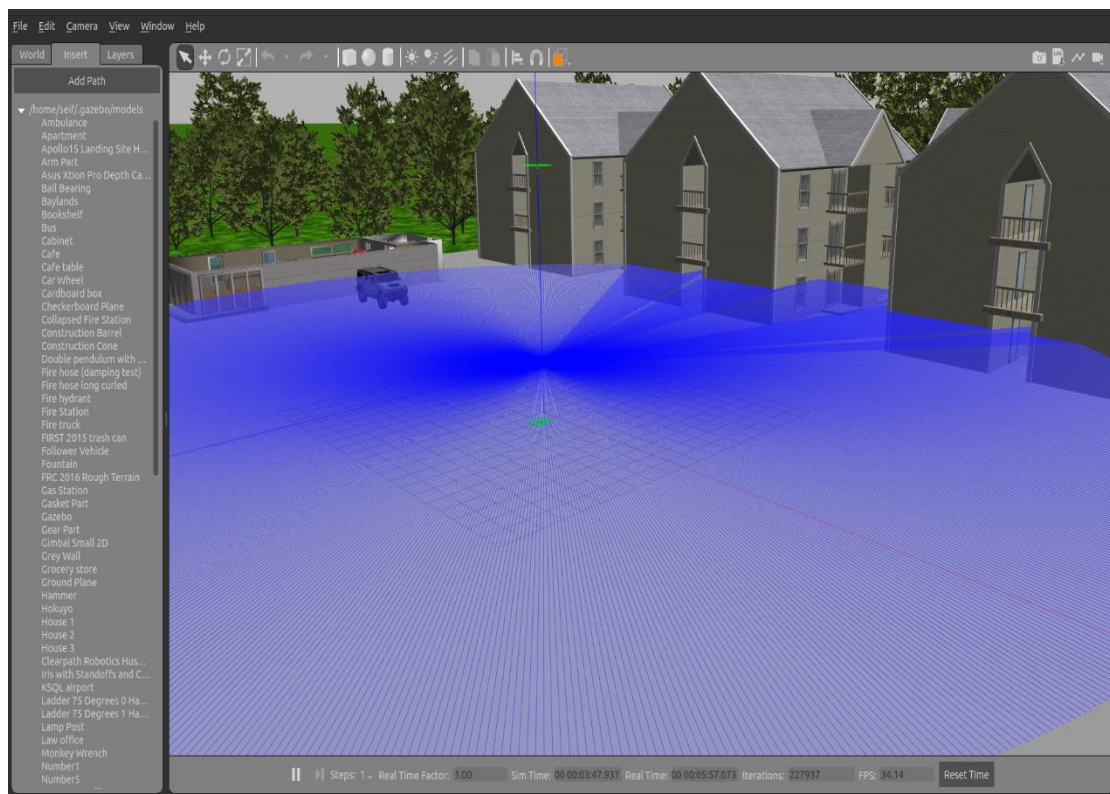


Figure 2: drone simulation

CHAPTER 4

FIRE DRONE SYSTEM

CHAPTER 4: FIRE DRONE SYSTEM

4.1.Overview

System architecture refers to the organization and arrangement of the components that make up a system, including hardware, software, and communication protocols. In the context of robotics, system architecture is particularly important for designing and implementing complex robot systems with multiple sensors, actuators, and control systems. A typical robot system architecture consists of several layers, including:

- **Perception Layer:** This layer includes the sensors and perception algorithms used to sense the environment and detect objects, obstacles, and other relevant information.
- **Planning and Control Layer:** This layer includes the algorithms used to plan and execute the robot's movements, such as motion planning, trajectory generation, and feedback control.
- **Communication Layer:** This layer includes the communication protocols used to exchange information between the different components of the robot system, such as sensor data, control commands, and status updates.
- **Hardware Layer:** This layer includes the physical components of the robot system, such as motors, actuators, and sensors.

These layers are typically implemented using a combination of hardware and software components, such as microcontrollers, embedded systems, and software libraries. The specific implementation of the system architecture will depend on the specific requirements of the robot system, such as the complexity of the environment, the number and type of sensors and actuators, and the desired level of autonomy.

In addition to these layers, some robot systems may also include higher-level components, such as decision-making algorithms, human-machine interfaces, and machine learning algorithms. These components can be used to enhance the robot's capabilities and adaptability in complex and changing environments. Overall, designing an effective robot system architecture requires careful consideration of the specific requirements and constraints of the robot application, as well as the interactions between the different components of the system.

A well-designed system architecture can help to ensure that the robot system is reliable, efficient, and adaptable to changing conditions, and can enable the development of more advanced and capable robot systems.

4.2. Hardware:

This section describes the hardware employed in this project and their configurations including the body of the drone and There are general components of firefighting drones including: Frame and Structure, Motors and Propellers, Power Source, Control System, Thermal Imaging Camera, Optical Camera, GPS Module, Communication System, Payload Delivery System, Obstacle Avoidance Sensors, Cooling System, Autonomous Flight Software, Lighting, Protective Enclosures and Table 1 shows the drone components:

Table 1: List of drone hardware components

1) APM 2.8
2) Raspberry pi 4 4GB
3) 5MP camera v1.3
4) ESC 30A
5) Brushless motor 2400kv
6) Propellers 10x4.7
7) Dual-side prototype PCB
8) Buck converter 3A
9) 3S LIPO battery 5200mAh
10) 3D printed f550 hexa-copter frame
11) Battery connector
12) Wires 2mm

4.2.1.APM Wiring Diagram:

The diagram below is an overview of how an APM2 can be connected to a receiver, ESC, GPS and even a camera gimbal. Figure 3 shows a typical APM2.8 autopilot in the market. APM2.8 has some advantages, such as offering advanced diagnostic tools that streamline troubleshooting and reduce resolution times, enhancing overall application reliability. It also provides comprehensive performance metrics, enabling better resource management and optimized user experiences. In this project, APM 2.8 is a professional quality IMU autopilot that is based on the Arduino Mega platform, and it enhances flight stability and precise navigation, crucial for accurate aerial surveillance in hazardous conditions. It also enables real-time data transmission, allowing for immediate assessment and response during firefighting.

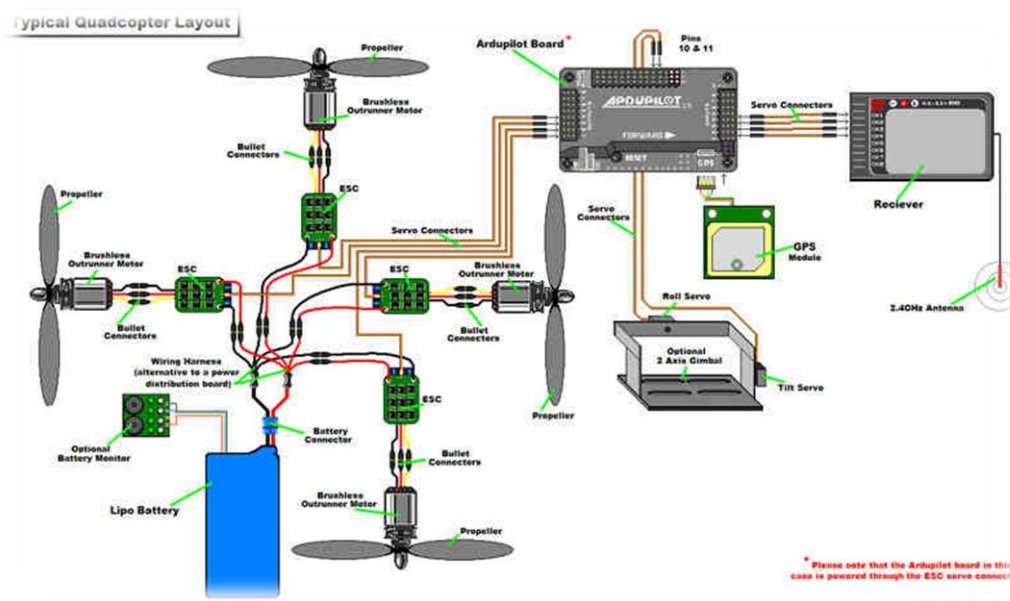


Figure 3: A typical APM2.8 autopilot in the market.

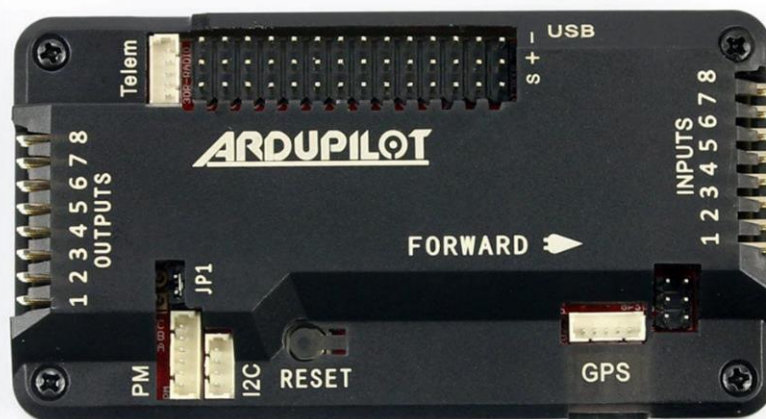


Figure 4: APM 2.8

4.2.2. Raspberry pi

Figure 5 shows The Raspberry Pi 4. A Raspberry Pi can be used for a wide range of applications. Making your Raspberry Pi into a retro arcade machine, using it as a web server, or using it as the brain for a robot, security system, IoT device, or dedicated Android device are all popular uses. In this project Using a Raspberry Pi 4 4GB in a fire drone allows for robust onboard processing capabilities, supporting advanced imaging and data analysis during flight. It also provides flexibility for integrating various sensors and modules, enhancing the drone's functionality in firefighting operations.



Figure 5 :Raspberry pi 4 4GB

4.2.3. Camera 1.3v

Figure 6 shows 5MP camera v1.3 which offers high-quality images and video with easy integration for Raspberry Pi projects. Using a 5MP Camera V1.3 in a fire drone enables high-resolution imaging, essential for detailed aerial assessments of fire scenes. It also offers reliable performance in various lighting conditions, ensuring consistent data capture during firefighting operations.

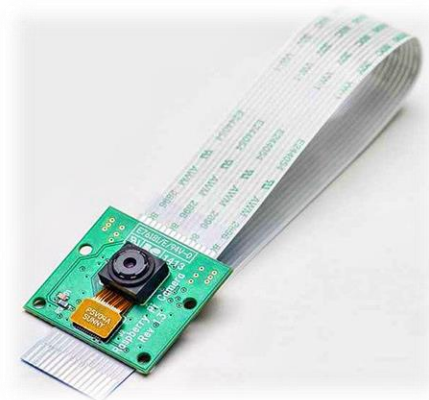


Figure 6: 5MP camera 1.3v

4.2.4. ESC 30A

Figure 7 shows The ESC 30A (Electronic Speed Controller) in fire drones regulates the speed and direction of the motors, ensuring precise control and stability during flight. It allows the drone to carry out complex maneuvers and maintain stability in turbulent conditions, which is crucial for navigating through fire and smoke. Benefits include enhanced flight performance, improved motor efficiency, and increased safety in fire emergency operations.

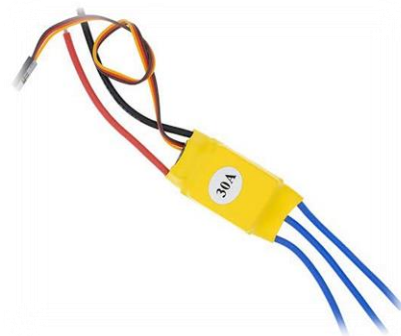


Figure 7: ESC 30A

4.2.5. Brushless Motor

Figure 8 shows Brushless Motor 2400KV which is a high-speed motor designed for efficiency and durability, providing 2400 RPM per volt of input. It's commonly used in applications requiring high performance and reliability, such as drones and RC vehicles. This motor offers low maintenance, high torque, and better heat dissipation compared to brushed motors. When using these motors in fire drones it provides high efficiency and power, essential for stable and responsive flight in challenging conditions. Its durability and reliability ensure consistent performance during extended firefighting missions.



Figure 8: Brushless motor 2400kv

4.2.6. propellers

Figure 9 shows 10x4.7 propellers which enhance lift and stability, ensuring smooth and controlled flight during firefighting operations.



Figure 9: Propellers 10x4.7

4.2.6. Dual-side prototype

Figure10 shows Dual-side prototype PCB which is a printed circuit board with conductive pathways on both sides, allowing for more complex circuit designs and increased component density. In this project dual-side prototype PCB enables more complex circuitry and efficient component layout, enhancing the drone's functionality and performance It allows components to be mounted on both sides of the

panel, optimizing space and functionality. it interconnects essential components like ESCs, sensors, and communication modules efficiently. This setup enhances the drone's performance by ensuring reliable electrical connections and compact design integration.

4.2.7. Buck converter

Figure 11 shows 3A Buck converter which steps down a higher DC voltage to a lower, stable voltage with high efficiency, handling up to 3 amps of current. It ensures reliable power delivery to sensitive electronic components, minimizing heat and energy loss. Its compact design makes it ideal for space-constrained applications like drones and other portable devices. It is used in the fire drones to ensure efficient power management for critical operations and steps down the voltage from the battery to a lower, stable voltage required by onboard electronics, ensuring efficient power.

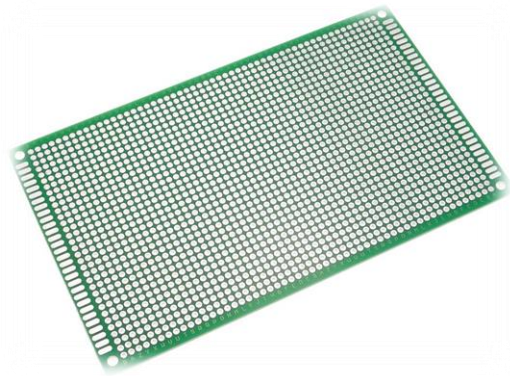


Figure 10: Dual-side prototype PCB.



Figure 11: Buck converter 3A.

4.2.8. LIPO battery

Figure 12 shows 3S LIPO battery 5200mAh and the 3S LIPO battery is a common lithium polymer alternative for RC hobbies and electronics. The “3S” designation refers to the battery pack's three series-connected cells. The overall voltage is 11.1 volts (3 cells x 3.7 volts), with each cell carrying a nominal value of 3.7 volts. The pros of a 3S LiPo battery (5200mAh) include a good balance between capacity and weight, suitable for medium-sized RC vehicles and drones. It also offers a lower cost compared to higher cell count packs, making it a cost-effective choice for many hobbyists. In fire drones Integrating a 3S LiPo battery (5200mAh) ensures robust power delivery suitable for extended flight durations, crucial in accessing remote or hazardous fire zones. Its manageable weight-to-capacity ratio supports efficient payload carrying capacity and enhances operational flexibility, vital for deploying firefighting measures swiftly and effectively.



Figure 12: 3S lipo battery 5200mAh.

4.2.9. f550 hexa-copter frame

Figure 13 shows 3D printed f550 hexa-copter frame Utilizing a 3D printed F550 hexacopter frame in the fire drones offers lightweight yet durable construction, ideal for maneuverability and resilience in challenging environments. Its customizable design facilitates integration of specialized equipment such as thermal cameras or payload systems, enhancing situational awareness and operational efficiency during fire response missions. The material flexibility of 3D printing allows for rapid prototyping and iterative improvements tailored to specific firefighting needs.



Figure 13: 3D printed f550 hexa-copter frame.

4.3.Implementation of Hardware

We also Implement a 3A- battery connector in fire drones to ensure reliable power distribution for sustained performance and Deploying 2mm wires ensures efficient power transmission and minimal weight, crucial for enhancing agility and reliability. we also use a 30A ESCs which enhances the fire drone's motor control precision and reliability under high-demand firefighting conditions (Figure 7). Figure 14 shows the drone.



Figure 14: Drone design.

4.4 Circuits

Figure 15 shows the bottom panel responsible for assembling all the ESC terminals and connecting them to the battery.

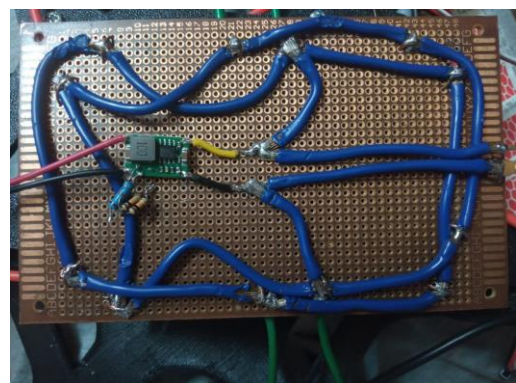


Figure 15: The bottom panel

CHAPTER 5

Computer Vision Techniques

CHAPTER 5: Machine Learning Algorithms

5.1 Overview

Object detection is a technique used in computer vision for the identification and localization of objects within an image or a video. Image Localization is the process of identifying the correct location of one or multiple objects using bounding boxes, which correspond to rectangular shapes around the objects.

This process is sometimes confused with image classification or image recognition, which aims to predict the class of an image or an object within an image into one of the categories or classes. You Only Look Once (YOLO) is a state-of-the-art, real-time object detection algorithm introduced in 2015 by Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi in their famous research paper <https://arxiv.org/abs/1506.02640> “You Only Look Once: Unified, Real-Time Object Detection”. The authors frame the object detection problem as a regression problem instead of a classification task by spatially separating bounding boxes and associating probabilities to each of the detected images using a single convolutional neural network (CNN).

YOLO achieved state-of-the-art results, beating other real-time object detection algorithms by a large margin. While algorithms like Faster RCNN work by detecting possible regions of interest using the Region Proposal Network and then performing recognition on those regions separately, YOLO performs all of its predictions with the help of a single fully connected layer. Some of the reasons why YOLO is leading the competition include its:

- i) Speed
- ii) Detection accuracy
- iii) Good generalization
- iv) Open source

5.2 Pre-Processing the data

5.2.1. Preparing the dataset

Probably the most time-consuming part of the experiment consisted of the data gathering and the pre-processing. An essential and inextricable task for every training model in machine learning or deep learning is the collection of the required dataset (collection of data).

5.2.1.1. Data gathering

We gathered fire images from website on google e.g. Unsplash & iStock. This data at the beginning was about 3000 fire image.

5.2.1.2. Annotation

A time-consuming but fundamental part of the pre-processing is the annotation of the images. To annotate the images, we use a tool named LabelImg, available for several platforms, enabling us an easy draw of the desired bounding box along with the annotation for each box, as seen in the following figure. LabelImg also offers us the opportunity for saving the annotation with either YOLO or PascalVOC format and we can make annotation with Roboflow.

5.2.1.3. Data augmentation

After finishing annotation up, the dataset has been uploaded up to colap to be trained. To be honest, at this stage the accuracy was not good enough so we had to gather more data and this large data was really time-consuming so, we resorted to a great technique called automatic annotation.

5.2.1.4. Roboflow



Figure 16: Roboflow

Roboflow is an end-to-end computer vision platform that provides tools for developers to build and deploy computer vision models. It's designed to streamline the process of creating datasets, training models, and deploying them to production. The platform is used by over 500,000 engineers and supports a wide range of functionalities. The low accuracy that we suffered from make it a must to use Roboflow. It helps a lot as it contains hundreds of datasets. Using about 5 or 6 of them has raised the accuracy hence using automatic annotation solved the time-consuming problem.

5.3. Why YOLO?

The difference between pre-trained models and yolo lies in their purpose, architecture, and training methodology

5.3.1.Purpose and Scope

Pre-trained Model: These refer to models that have been trained on a large dataset (often a general-purpose dataset like ImageNet) for a specific task (e.g., image classification, object detection) Pre-trained models are trained by experts and are made available for transfer learning, where they can be fine-tuned on a smaller, task-specific dataset.

5.4. YOLO

YOLO specifically refers to an instance of the YOLO (You Only Look Once) object detection model. YOLO models are designed for real-time object detection tasks, where the goal is to localize and classify objects in images or videos efficiently.

5.4.1. Usage

Pre-trained Models are often used as starting points for specific tasks. By leveraging pre-trained weights, developers can save significant training time and computational resources when fine-tuning models on new datasets. YOLO It is used specifically for real-time object detection applications where speed and accuracy are crucial. YOLO models are deployed in scenarios such as video surveillance, autonomous vehicles, and industrial automation.

In summary, pre-trained models are more general and can be applied to various tasks after fine-tuning, whereas YOLO is a specific model architecture optimized for real-time object detection tasks.

5.5.YOLO Algorithm

5.5.1. YOLO Architecture

The YOLO algorithm takes an image as input and then uses a simple deep convolutional neural network to detect objects in the image. The architecture of the CNN model that forms the backbone of YOLO is shown below.

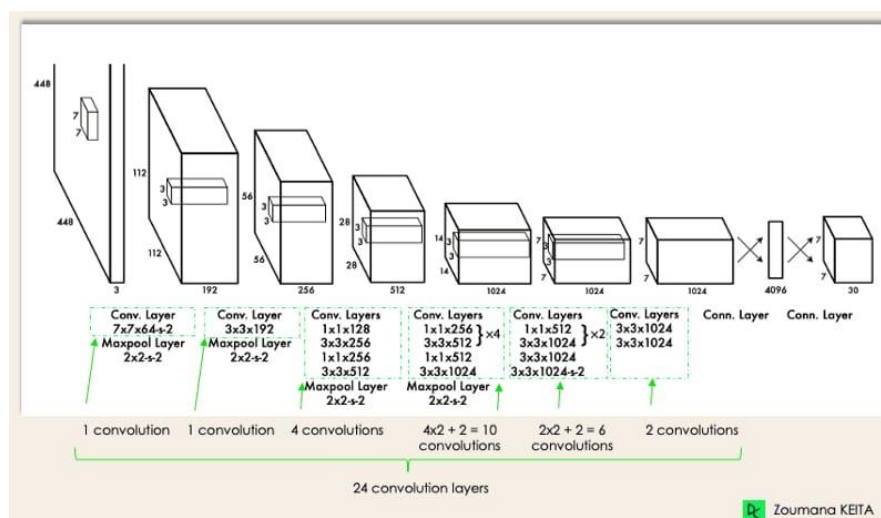


Figure 17: CNN-Architecture

The first 20 convolution layers of the model are pre-trained using ImageNet by plugging in a temporary average pooling and fully connected layer. Then, this pre-trained model is converted to perform detection since previous research showcased that adding convolution and connected layers to a pretrained network improves performance. YOLO's final fully connected layer predicts both class probabilities and bounding box coordinates.

5.5.2 How YOLO works?

The team that developed the algorithm apply a single neural network to the full image. This network divides the image into regions and predicates bounding boxes and probabilities for each region. These bounding boxes are weighted by the predicated probabilities. The image below shows that.

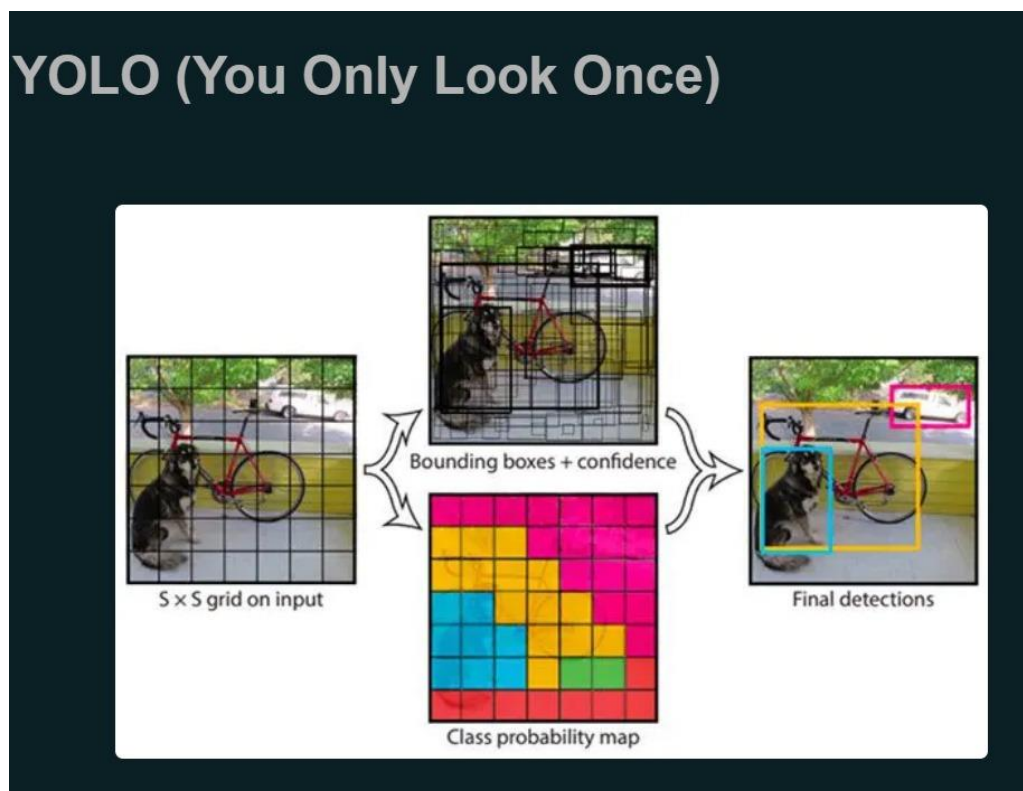


Figure 18: YOLO-bounding box.

5.5.3 YOLOv8

YOLOv8 is the newest state-of-the-art YOLO model that can be used for object detection, image classification, and instance segmentation tasks. YOLOv8 was developed by Ultralytics, who also created the influential and industry-defining YOLOv5 model. YOLOv8 includes numerous architectural and developer experience changes and improvements over YOLOv5. YOLOv8 is under active development as of writing this documentation, as Ultralytics work on new features and respond to feedback from the community. Indeed, when Ultralytics releases a model, it enjoys long-term support: the organization works with the community to make the model the best it can be.

Here are a few main reasons why we consider using YOLOv8 for our computer vision project:

- i) YOLOv8 has a high rate of accuracy measured by Microsoft COCO and Roboflow 100.
- ii) YOLOv8 comes with a lot of developer-convenience features, from an easy-to-use CLI to a well-structured Python package.
- iii) There is a large community around YOLO and a growing community around the YOLOv8 model, meaning there are many people in computer vision circles who may be able to assist you when you need guidance.

5.6.IDE

An integrated development environment (IDE) is a software application that provides comprehensive facilities for software development. An IDE normally consists of at least a source-code editor, build automation tools, and a debugger. Some IDEs, such as IntelliJ IDEA, Eclipse and Lazarus contain the necessary compiler, interpreter or both; others, such as SharpDevelop and NetBeans.

5.6.1. Jupiter Notebook

It is a web-based notebook environment for interactive computing in which you can combine code execution, rich text, mathematics, plots and rich media.

Jupyter Notebooks are primarily used by data professionals, particularly data analysts and data scientists. According to the Kaggle Survey 2022 results, Jupyter Notebooks are the most popular data science IDE, used by over 80% of respondents.



Figure 19: Jupyter-dashboard.

```
from keras.callbacks import Callback, LambdaCallback\n\nranges = {\n    "Rotation": np.arange(0, 31, 1),\n    "Width Shift": np.arange(0, 0.21, 0.1),\n    "Height Shift": np.arange(0, 0.21, 0.1),\n    "Zoom": np.arange(0, 0.21, 0.1),\n    "Shear": np.arange(0, 0.21, 0.1),\n    "Horizontal Flip": [True, False],\n    "Vertical Flip": [True, False],\n    "Optimizer": [Adam(), Nadam(), RMSprop(), Adadelta(), Adagrad(), SGD()],\n    "Batch Size": [8, 16, 32, 64],\n    "TL Learn Ratio": np.arange(0, 26, 1),\n}\n\nSOLUTION_SIZE = len(ranges.keys())\n\nIn [19]: # Population Initialization\npopulation = np.random.uniform(\n    low=LOWER_BOUND,\n    high=UPPER_BOUND,\n    size=(POPULATION_SIZE, SOLUTION_SIZE)\n)\nprint(population.shape)\nprint(population[0])\n\n(5, 10)\n[0.81767134 0.19115407 0.44148603 0.03985403 0.05342278 0.61116924\n 0.42330831 0.94474926 0.94003813 0.58069588]
```

Figure 20: Jupyter notebook

A notebook integrates code and its output into a single document that combines visualizations, narrative text, mathematical equations, and other rich media. In other words: it's a single document where you can run code, display the output, and add explanations, formulas, charts, and make your work more transparent, understandable, repeatable, and shareable.

5.6.2. Google Colap

Google Colab is a cloud-based Jupyter notebook environment from Google Research. With its simple and easy-to-use interface, Colab helps you get started with your data science journey with almost no setup.

If you're interested in data science with Python, Colab is a great place to kickstart your data science projects without worrying about configuring your environment. Google Colab facilitates writing and execution of Python code right from your browser, and also comes with some of the most popular Python data science libraries pre-installed.

5.6.3. Programming

The main language used in this project is Python. Python's open-source libraries are not the only feature that make it favorable for machine learning and AI tasks. Python is also highly versatile and flexible, meaning it can also be used alongside other programming languages when needed. Even further, it can operate on nearly all OS and platforms on the market. Implementing deep neural networks and machine learning algorithms can be extremely time consuming, but Python offers many packages that cut down on this. It is also an object-oriented programming (OOP) language, which makes it extremely useful for efficient data use and categorization.

5.6.4. libraries

5.6.4.1 OpenCV

OpenCV is one of the image processing libraries in python. openCV provides a wide range of computer vision algorithms and functions, including image and video processing, feature detection, object recognition, machine learning, and more. It has a large community of developers and users, making it a popular choice for computer vision projects. The image processing library provides access to over 2,500 state-of-the-art and classic algorithms. Users can use OpenCV to perform several specific tasks like removing red eyes and following eye movements. Some of the features of OpenCV include:

- i) Image and video capture and processing.
- ii) Object detection and tracking.
- iii) Facial recognition and detection.
- iv) Optical character recognition (OCR).
- v) Machine learning algorithms for image and data analysis.
- vi) 3D reconstruction and augmented reality

OpenCV is easy to install and use. It comes with pre-built libraries and is compatible with multiple platforms such as Windows, Linux, macOS, iOS, and Android. It also has bindings for various programming languages like Python, Java, and MATLAB.

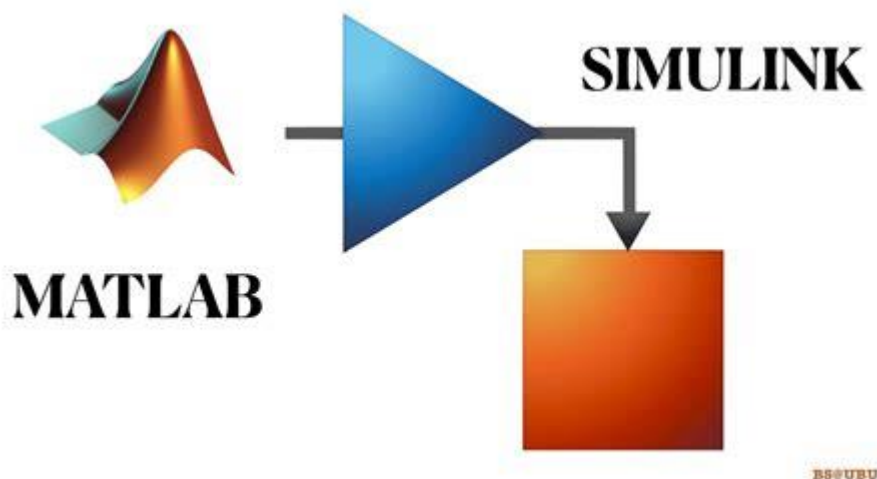


Figure 21: MATLAB

5.6.4.2. Ultralytics

The Ultralytics library is significant in the field of computer vision and artificial intelligence, particularly for tasks involving object detection, tracking, and analysis. The library is known for its implementation of the YOLO (You Only Look Once) models, which are state-of-the-art for real-time object detection. Here are some key points about the Ultralytics library and its importance

- i) **State-of-the-Art Models:** Ultralytics provides cutting-edge models like YOLOv8, which is known for its speed, accuracy, and ease of use. It's suitable for a wide range of tasks including object detection, multi-object tracking, instance segmentation, pose estimation, and image classification¹.

- ii) Ease of Use: The library offers a user-friendly interface and comprehensive documentation making it accessible for both beginners and experienced practitioners¹.
- iii) Interactive Notebooks: Ultralytics provides interactive notebooks paired with YouTube tutorials, which help users learn and implement advanced features of YOLOv8 effectively¹.
- iv) Pretrained Models: It offers pretrained models on datasets like COCO and ImageNet, which can be used for detection, segmentation, and classification tasks, saving time and resources in model training¹.
- v) Integration with AI Platforms: The library integrates with leading AI platforms to enhance tasks like dataset labeling, training, visualization, and model management. This includes collaborations with platforms like Roboflow, ClearML, Comet, Neural Magic, and OpenVINO.

```

$ pip install ultralytics
$ pip install roboflow

Collecting ultralytics
  Downloading ultralytics-6.2.41-py3-none-any.whl (792 kB)
Requirement already satisfied: numpy<2.0.0,>=1.23.5 in /usr/local/lib/python3.10/dist-packages (from ultralytics) (1.25.2)
Requirement already satisfied: matplotlib>3.3.0 in /usr/local/lib/python3.10/dist-packages (from ultralytics) (3.7.1)
Requirement already satisfied: opencv-python>4.6.0 in /usr/local/lib/python3.10/dist-packages (from ultralytics) (4.8.0.76)
Requirement already satisfied: pillow>8.3.0 in /usr/local/lib/python3.10/dist-packages (from ultralytics) (9.4.0)
Requirement already satisfied: pyyaml>5.1 in /usr/local/lib/python3.10/dist-packages (from ultralytics) (6.0.1)
Requirement already satisfied: requests>2.23.0 in /usr/local/lib/python3.10/dist-packages (from ultralytics) (2.31.0)
Requirement already satisfied: scipy>1.4.1 in /usr/local/lib/python3.10/dist-packages (from ultralytics) (1.11.4)
Requirement already satisfied: torch>1.8.0 in /usr/local/lib/python3.10/dist-packages (from ultralytics) (2.3.0+cu121)
Requirement already satisfied: torchvision>0.9.0 in /usr/local/lib/python3.10/dist-packages (from ultralytics) (0.18.0+cu121)
Requirement already satisfied: tqdm>4.64.0 in /usr/local/lib/python3.10/dist-packages (from ultralytics) (4.66.4)
Requirement already satisfied: psutil in /usr/local/lib/python3.10/dist-packages (from ultralytics) (5.9.5)
Requirement already satisfied: py-cpuinfo in /usr/local/lib/python3.10/dist-packages (from ultralytics) (9.0.0)
Requirement already satisfied: pandas>1.4 in /usr/local/lib/python3.10/dist-packages (from ultralytics) (2.0.3)
Requirement already satisfied: seaborn>0.11.0 in /usr/local/lib/python3.10/dist-packages (from ultralytics) (0.13.1)
Collecting ultralytics-thop>2.0.0 (from ultralytics)
  Downloading ultralytics_thop-2.0.0-py3-none-any.whl (25 kB)
Requirement already satisfied: contourpy>1.0.1 in /usr/local/lib/python3.10/dist-packages (from matplotlib>3.3.0->ultralytics) (1.2.1)
Requirement already satisfied: cycler>0.10 in /usr/local/lib/python3.10/dist-packages (from matplotlib>3.3.0->ultralytics) (0.12.1)
Requirement already satisfied: fonttools>4.22.0 in /usr/local/lib/python3.10/dist-packages (from matplotlib>3.3.0->ultralytics) (4.53.0)
Requirement already satisfied: kiwisolver>1.0.1 in /usr/local/lib/python3.10/dist-packages (from matplotlib>3.3.0->ultralytics) (1.4.5)
Requirement already satisfied: packaging>20.0 in /usr/local/lib/python3.10/dist-packages (from matplotlib>3.3.0->ultralytics) (24.1)
Requirement already satisfied: pyparsing>2.3.1 in /usr/local/lib/python3.10/dist-packages (from matplotlib>3.3.0->ultralytics) (3.1.2)
Requirement already satisfied: python-dateutil>2.7 in /usr/local/lib/python3.10/dist-packages (from matplotlib>3.3.0->ultralytics) (2.8.2)
Requirement already satisfied: pytz>2020.1 in /usr/local/lib/python3.10/dist-packages (from pandas>1.4->ultralytics) (2022.6)
Requirement already satisfied: xdata>2022.1 in /usr/local/lib/python3.10/dist-packages (from pandas>1.4->ultralytics) (2024.1)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.10/dist-packages (from requests>2.23.0->ultralytics) (3.3.2)
Requirement already satisfied: idna<4,>=2.3 in /usr/local/lib/python3.10/dist-packages (from requests>2.23.0->ultralytics) (3.7)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.10/dist-packages (from requests>2.23.0->ultralytics) (2.0.7)
Requirement already satisfied: certifi>2017.4.17 in /usr/local/lib/python3.10/dist-packages (from requests>2.23.0->ultralytics) (2024.6.2)
Requirement already satisfied: filelock in /usr/local/lib/python3.10/dist-packages (from torch>1.8.0->ultralytics) (3.15.3)
Requirement already satisfied: torchvision>0.9.0 in /usr/local/lib/python3.10/dist-packages (from torchvision>0.9.0->ultralytics) (0.18.0+cu121)

```

Figure22: Importing Ultralytics and roboflow.

```

from IPython import display
import ultralytics
import os
from ultralytics import YOLO
from IPython.display import Image

[3] HOME=os.getcwd()
print(HOME)

/content

[4] from roboflow import Roboflow
rf = Roboflow(api_key="K8Sg6SCTf8xyIvnp8R")
project = rf.workspace("rahan-2vlay").project("fire-detection-vdtec")
dataset = project.version(1).download("yolov8")

loading Roboflow workspace...
loading Roboflow project...
Dependency ultralytics==8.0.196 is required but found version=8.2.41, to fix: 'pip install ultralytics==8.0.196'
Downloading Dataset Version Zip in Fire-Detection-1 to yolov8: 100%|#####| 36373/36373 [00:00<00:00, 45081.65it/s]
Extracting Dataset Version Zip to Fire-Detection-1 in yolov8: 100%|#####| 1514/1514 [00:00<00:00, 6868.06it/s]

[5] !yolo task=detect mode=train model=yolov8n.pt data={dataset.location}/data.yaml epochs=30 imgsz=640

Downloading https://github.com/ultralytics/assets/releases/download/v8.2.0/yolov8n.pt to 'yolov8n.pt'...
100% 6.23M/6.23M [00:00<00:00, 11000/s]
Ultralytics YOLOv8 2.41 Python 3.10.12 torch 2.3.0cu121 CUDA:0 (Tesla T4, 15102MiB)
engine/trainer: task=detect, mode=train, model=yolov8n.pt, data=/content/Fire-Detection-1/data.yaml, epochs=30, time=None, patience=100, batch=16, imgsz=640, save=True, save_period=1, cache=False
Downloading https://ultralytics.com/assets/arial.ttf to '/root/.config/Ultralytics/arial.ttf'...
100% 755k/755k [00:00<00:00, 25.90M/s]
Overriding model.yaml nc=80 with nc=1

Completed at 3:47 AM

```

Figure23: Lunning data from Roboflow to Colab.

```

!yolo task=detect mode=train model=yolov8n.pt data={dataset.location}/data.yaml epochs=30 imgsz=640

Downloading https://github.com/ultralytics/assets/releases/download/v8.2.0/yolov8n.pt to 'yolov8n.pt'...
100% 6.23M/6.23M [00:00<00:00, 11000/s]
Ultralytics YOLOv8 2.41 Python 3.10.12 torch 2.3.0cu121 CUDA:0 (Tesla T4, 15102MiB)
engine/trainer: task=detect, mode=train, model=yolov8n.pt, data=/content/Fire-Detection-1/data.yaml, epochs=30, time=None, patience=100, batch=16, imgsz=640, save=True, save_period=1, cache=False
Downloading https://ultralytics.com/assets/arial.ttf to '/root/.config/Ultralytics/arial.ttf'...
100% 755k/755k [00:00<00:00, 25.90M/s]
Overriding model.yaml nc=80 with nc=1

from n params module arguments
0 -1 1 464 ultralytics.nn.modules.conv.Conv [3, 16, 3, 2]
1 -1 1 4672 ultralytics.nn.modules.conv.Conv [16, 32, 3, 2]
2 -1 1 7360 ultralytics.nn.modules.block.C2F [32, 32, 1, True]
3 -1 1 18848 ultralytics.nn.modules.conv.Conv [32, 64, 3, 2]
4 -1 2 49664 ultralytics.nn.modules.block.C2F [64, 64, 2, True]
5 -1 1 73984 ultralytics.nn.modules.conv.Conv [64, 128, 3, 2]
6 -1 2 197632 ultralytics.nn.modules.block.C2F [128, 128, 2, True]
7 -1 1 264424 ultralytics.nn.modules.conv.Conv [128, 256, 3, 2]
8 -1 1 460288 ultralytics.nn.modules.block.C2F [256, 256, 1, True]
9 -1 1 164608 ultralytics.nn.modules.block.SPPF [256, 256, 5]
10 -2 2 0 torch.nn.modules.upstreaming.Upstream [None, 2, 'nearest']
11 [-1, 6, 1] 0 ultralytics.nn.modules.conv.Concat [1]
12 -1 1 148224 ultralytics.nn.modules.block.C2F [384, 128, 1]
13 -1 1 0 torch.nn.modules.upstreaming.Upstream [None, 2, 'nearest']
14 [-1, 4, 1] 0 ultralytics.nn.modules.conv.Concat [1]
15 -1 1 37248 ultralytics.nn.modules.block.C2F [192, 64, 1]
16 -1 1 36992 ultralytics.nn.modules.conv.Conv [64, 64, 3, 2]
17 [-1, 12, 1] 0 ultralytics.nn.modules.conv.Concat [1]
18 -1 1 123648 ultralytics.nn.modules.block.C2F [192, 128, 1]
19 -1 1 147712 ultralytics.nn.modules.conv.Conv [128, 128, 3, 2]
20 [-1, 9, 1] 0 ultralytics.nn.modules.conv.Concat [1]
21 -1 1 493056 ultralytics.nn.modules.block.C2F [384, 256, 1]
22 [15, 16, 21] 1 751507 ultralytics.nn.modules.head.Detect [1, [64, 128, 256]]

Model summary: 225 layers, 3011843 parameters, 3011027 gradients, 8.2 GFLOPs

Transferred 319/355 items from pretrained weights

Completed at 3:47 AM

```

Figure24: Adam optimizer.

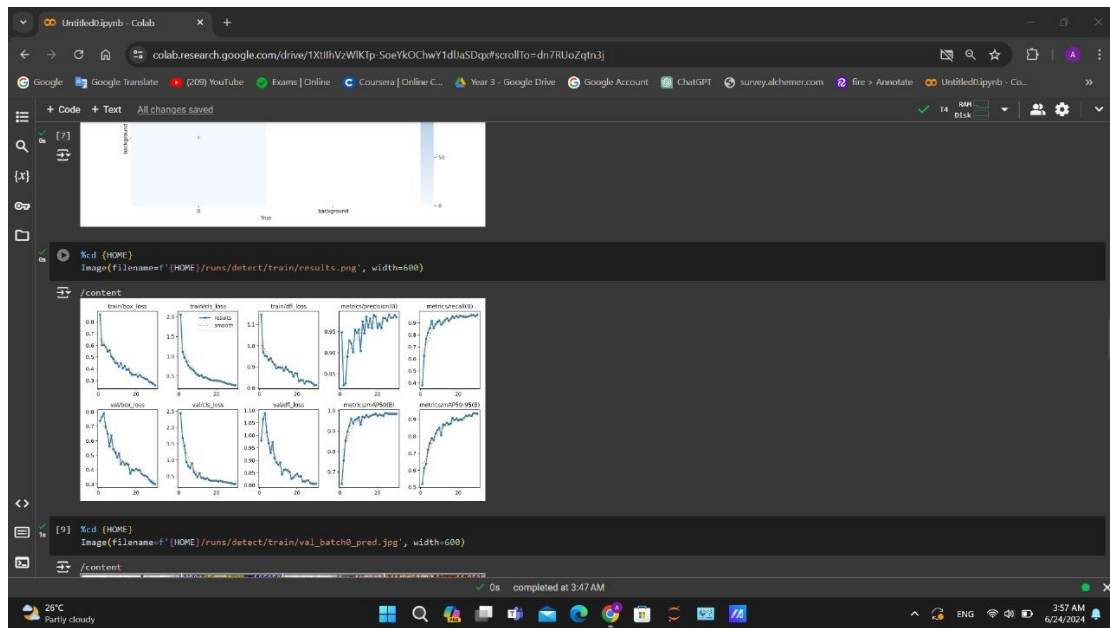


Figure25: The result



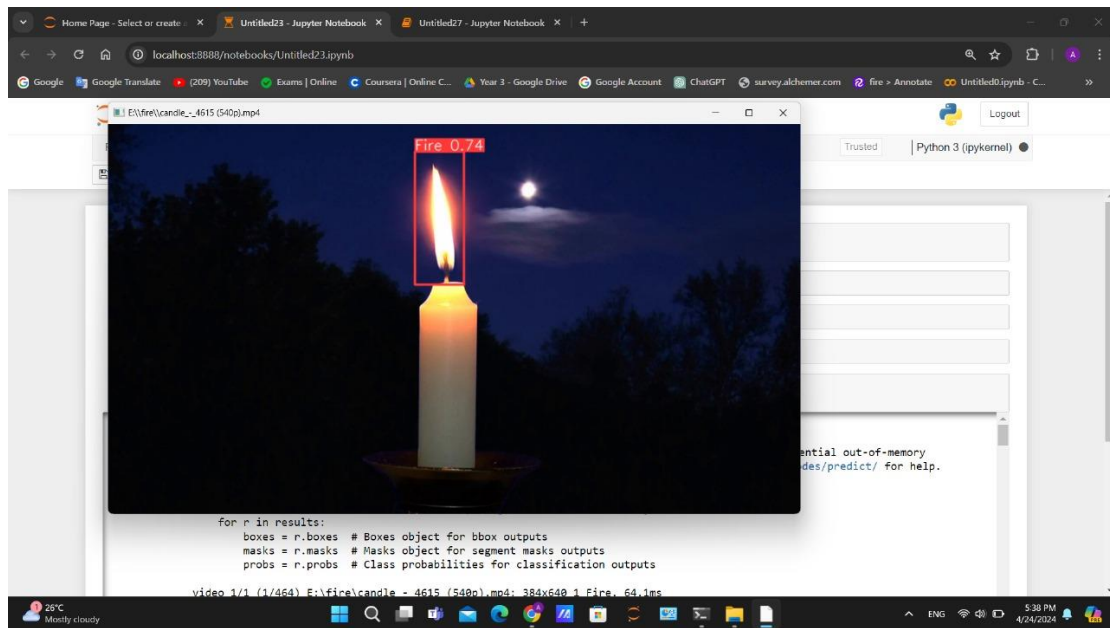


Figure26:Final result of detection

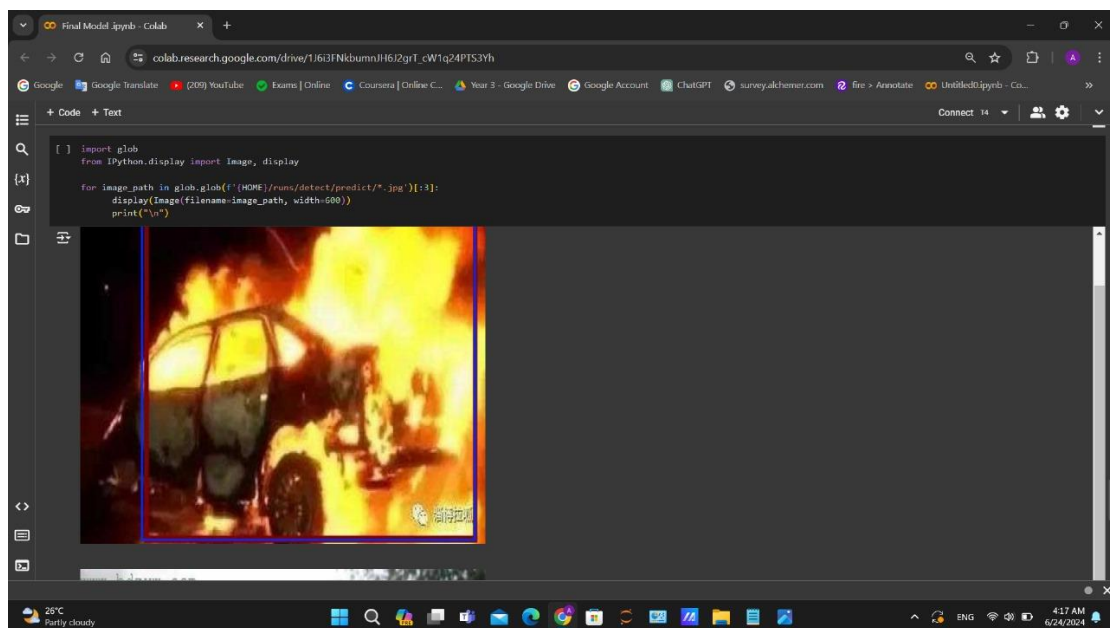
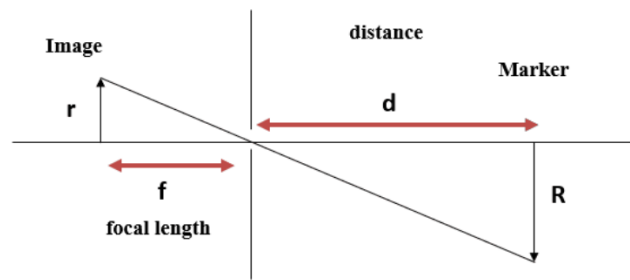


Figure27: validation and testing,

5.7. Distance Calculation

To calculate the distance between the fire (detected object) and the raspberry pi camera we will use the Triangle similarity method. We show here one specific figure of the distance between an object and a marker to understand the triangle similarity method better using the following different parameters:



In order to determine the distance from a camera to a known object or marker, we're going to utilize triangle similarity. As we can see in this figure, we have:

f : focal length

r : radius of the marker in the image plane

R : radius of the marker in the object plane

d : the unknown parameter, distance from the camera to the object

We consider the camera generates a one-to-one relationship between the object and the image. Then, we can deduce a relationship between known parameters:

$$\frac{f}{d} = \frac{1}{R}$$

Figure28:the equation for distance

The focal length here will always remain constant because we use raspberry pi camera module V1, and the focal length will equal 3.60mm The radius of the marker in the image plane (r) also will remain constant and it will equal 10 cm The radius of the marker in the object plane (R) will be calculated during live video streaming. So, we can determine the value of the distance (d) as follows:

$$d = \frac{f \cdot R}{r}$$

CHAPTER 6

LINKING BETWEEN FCU AND RASPBERRY PI

CHAPTER 6: linking between FCU and Raspberry Pi

6.1 Overview

A typical robot system architecture is often organized into several layers to facilitate modularity, flexibility, and efficient operation.

- i) **Hardware Layer:** This is the physical layer of the robot system, including actuators, sensors, processors, and communication interfaces. It's responsible for interacting with the physical environment.
- ii) **Device Abstraction Layer:** This layer abstracts the hardware details and provides a uniform interface for higher-level software components to interact with the hardware. It includes drivers and APIs for sensors, actuators, and other hardware components.
- iii) **Perception Layer:** This layer is responsible for processing sensory data from various sensors such as cameras, lidars, and depth sensors. It involves tasks like object detection, localization, mapping, and scene understanding.
- iv) **Decision-Making Layer:** Also known as the cognition or planning layer, this part of the architecture interprets the data from the perception layer and makes decisions based on it. It involves high-level reasoning, path planning, task scheduling, and decision-making algorithms.
- v) **Control Layer:** This layer translates high-level commands generated by the decision-making layer into low-level commands that can be directly executed by the actuators. It's responsible for motion control, stability, and coordination of robot movements.
- vi) **Communication Layer:** This layer handles communication between different components of the robot system, as well as communication with external systems such as human operators or other robots. It includes protocols for data exchange, synchronization, and coordination.
- vii) **Human-Robot Interface (HRI) Layer:** This optional layer provides interfaces for human interaction with the robot, including graphical user interfaces (GUIs), natural language interfaces, gesture recognition, and teleoperation capabilities.
- viii) **System Management Layer :** This layer handles system-level functions such as resource allocation, monitoring, fault detection and recovery, software updates, and system diagnostics. These layers are typically implemented using a combination of hardware and software components, such as microcontrollers, embedded systems, and software libraries. The specific implementation of the system architecture will depend on the specific requirements of the robot system, such as the complexity of the environment, the number and type of sensors and actuators, and the desired level of

autonomy. In addition to these layers, some robot systems may also include higher-level components, such as decision-making algorithms, human-machine interfaces, and machine learning algorithms. These components can be used to enhance the robot's capabilities and adaptability in complex and changing environments. Overall, designing an effective robot system architecture requires careful consideration of the specific requirements and constraints of the robot application, as well as the interactions between the different components of the system. A well-designed system architecture can help to ensure that the robot system is reliable, efficient, and adaptable to changing conditions, and can enable the development of more advanced and capable robot systems.

Table 2 components

Name
Raspberry Pi
Autopilot APM 2.8 flight controller
Jumper wires

6.2.Component

This part will describe (raspberry pi & autopilot APM 2.8 flight controller) and connections components between them.



Figure 29: Raspberry pi



Figure 30: Autopilot apm 2.8.

6.2.1.APM in Flight Controllers

ArduPilot Mega (APM) is a professional quality IMU autopilot that is based on the Arduino Mega platform. This autopilot flight controller can control fixed-wing aircraft, multi-rotor helicopters, as well as traditional helicopters

6.1.3Reasons Why APM 2.8 is Considered Outdated

- i) **Limited Processing Power:** The ATmega2560 microcontroller has limited processing capabilities compared to modern flight controllers, which use more powerful ARM Cortex processors.
- ii) **Sensor Technology:** The onboard sensors in APM 2.8 are less accurate and have slower update rates compared to newer sensors found in current flight controllers.
- iii) **Lack of Advanced Features:** Modern flight controllers offer advanced features like onboard logging, better integration with companion computers, and support for more sophisticated navigation algorithms.
- iv) **Firmware Support:** While the ArduPilot firmware still supports APM 2.8, newer versions of the firmware are optimized for more advanced hardware, providing better performance and additional features.
- v) **Power Consumption:** Newer flight controllers are more power-efficient, which is critical for UAVs where power consumption directly affects flight time.
- vi) **Size and Weight:** Advances in miniaturization have led to smaller and lighter flight controllers, which are beneficial for all types of UAVs, particularly small drones.

6.3.1. Power Connections

Connect the Raspberry Pi to a power source using a USB cable, connect the APM 2.8 to a power source using another USB cable. This powers the APM 2.8 and allows you to program it.

6.3.2. UART

Identify the UART pins on both the Raspberry Pi and the APM 2.8. Typically, these are labeled as TX (transmit) and RX (receive). Connect the TX pin of the Raspberry Pi to the RX pin of the APM 2.8. Connect the RX pin of the Raspberry Pi to the TX pin of the APM 2.8. Ensure that both devices share a common ground by connecting the ground (GND) pins of the Raspberry Pi and the APM 2.8.

6.3.3. UART (Serial) Connection:

UART Communication Between Raspberry Pi 4 and APM 2.8 Flight Controller in a Fire Drone Project: UART (Universal Asynchronous Receiver/Transmitter) is a hardware communication protocol that facilitates serial communication between devices. In a fire drone project, integrating a Raspberry Pi 4 with an APM 2.8 flight controller via UART can enable advanced control and data acquisition capabilities. Here's a detailed guide on how to set up and use UART communication between these two components.

Overview of UART Communication:

UART is a serial communication protocol that involves two primary lines:

- i) **TX (Transmit):** Sends data from one device.
- ii) **RX (Receive):** Receives data from another device.

Each device has its own TX and RX lines, which need to be cross-connected to establish communication.

1. Enabling UART on Raspberry Pi 4

- i) Open the terminal on the Raspberry Pi.
- ii) Run **sudo raspi-config**.
- iii) Navigate to **Interface Options** and select **Serial**.
- iv) Disable the login shell over serial and enable the serial port hardware.

2. Edit the Boot Configuration

- i) Edit `/boot/config.txt` using `sudo nano /boot/config.txt`.
- ii) Add or ensure the line `enable_uart=1` is present.
- iii) Save and exit (Ctrl+X, Y, Enter).

3. Reboot:

- i) Reboot the Raspberry Pi with **sudo reboot**.

6.2.4.4. Configuring the APM 2.8 Flight Controller:

1. Install Mission Planner (or a similar ground control software)
2. **Connect APM 2.8:**
 - i) Connect the APM 2.8 to your computer via USB.
 - ii) Open Mission Planner and connect to the APM 2.8.

Configure the serial port settings in the Mission Planner to match those of the UART settings you plan to use on the Raspberry Pi. and calibrate the drone

Establishing Communication:

1. Serial Port Access on Raspberry Pi:

- Identify the serial port, **/dev/ttyAMA0** Don't choose **/dev/ttyAMAC0** **/dev/ttyAMA0** is a device file that represents the serial port on the Raspberry Pi. Specifically, it is the primary UART (Universal Asynchronous Receiver/Transmitter) on the Raspberry Pi. Enabling **/dev/ttyAMA0** refers to enabling the serial communication interface on the Raspberry Pi, not USB.

Python Serial Communication:

There are versions of python like 3.9.11 and 2.8.16 but aurdopilot don't work with both of them so, we need virtual environment 2.8.13. There are libraries we use:

- **pyserial**
(is a Python library used for serial communication. It provides support for accessing and controlling serial ports on various platforms, making it essential for applications that require communication with serial devices like microcontrollers, GPS modules, and more.)

1.Dev

("Dev" is widely recognized in the software industry and is used colloquially in discussions, documentation, and conversations related to software development practices and methodologies)

2.Lxml

(is a Python library that provides a comprehensive toolkit for working with XML and HTML documents.)

3.OpenCV

OpenCV is a widely used open-source computer vision and machine learning software library. It provides a comprehensive set of functions and algorithms for real-time computer vision tasks, such as image processing, object detection, facial recognition, and more. OpenCV supports various programming languages, including C++, Python, and Java, making it accessible and versatile for developers working on computer vision application.

4,Pymavlink

Pymavlink is a low level and general purpose MAVLink message processing library, written in Python. It has been used to implement MAVLink communications in many types of MAVLink systems, including a GCS (MAVProxy), Developer APIs (DroneKit) and numerous companion computer MAVLink applications.

5.libxml2-dev

(The libxml2-dev package contains the development files, including headers and static libraries, necessary for compiling applications that use libxml2. This package is essential for developers who need to build software that interfaces with or depends on libxml2).

6. JTK

(wxPython is a popular library for creating graphical user interfaces (GUIs) in Python. It is a wrapper around the native GUI toolkit wxWidgets, which allows developers to create native applications for Windows, Mac, and Linux from a single codebase).

7.Matplotlib

(is a comprehensive library for creating static, animated, and interactive visualizations in Python. Matplotlib makes easy things easy and hard things possible. Create publication quality plots. Make interactive figures that can zoom, pan, update).

MAVProxy 1.5.2

A UAV ground station software package for MAVLink based system MAVProxy is a fully-functioning GCS for UAV's, designed as a minimalist, portable and extendable GCS for any autonomous system supporting the MAVLink protocol (such as one using ArduPilot). MAVProxy is a powerful command-line based “developer” ground station software. It can be extended via add-on modules, or complemented with another ground station, such as Mission Planner, APM Planner 2, QGroundControl etc, to provide a graphical user interface.

Common MAVproxy Commands:

alias	: command aliases: usage: alias <add remove list>
arm	: arm motors: usage: arm <check uncheck list throttle safetyon safetyoff>

auxopt	: select option for aux switches on CH7 and CH8 (ArduCopter only): Usage:
auxopt	set show reset list
disarm	: disarm motors
land	: auto land
log	: log file handling: usage: log <list download erase resume status cancel>
mode	: mode change
module	: module commands: usage: module <list load reload unload>
param	: parameter handling: Usage: param <fetch save set show load preload forceload diff download help>
position	: position: Usage: position x y z (meters)
rc	: RC input control: Usage: rc <channel all> <pwmvalue>
reboot	: reboot autopilot
repeat	: repeat a command at regular intervals: Usage: repeat <add remove clean>
script	: run a script of MAVProxy commands
setspeed	: do_change_speed: Usage: setspeed SPEED_VALUE
setyaw	: condition_yaw: Usage: yaw ANGLE ANGULAR_SPEED MODE:[0 absolute / 1 relative]
takeoff	: takeoff: Usage: takeoff ALTITUDE_IN_METERS

```

MAVProxy Vehicle Link Mission Rally Fence Parameter
STABILIZE ARM GPS:0(0) Vcc:5.00 Radio:- INS MAG AS RING AHRS EXF LOG FEN RC TERR PRE PWR: 5rv0.00
Link:1 OK 100.0% (837 pkts, 0 lost, 0.00s delay) Batt: 100%/12.60V 0.0A
Hdg:347/0 Alt:0m AGL:-- AirSpeed:0m/s GPSSpeed:0m/s Thr:0 Roll:0 Pitch:0 Wind:--
WP:0 Distance:0m Bearing:0 AltError:0m(L) AspError:0m/s(H) FlightTime:- ETR:- Param:1422/1422 Mission:--
online system 1
Mode:STABILIZE
AP: Ready SIM
AP: ArduCopter V4.6.0-dev (727f28db)
AP: 4ecba458dab046094412a08eef40c9e
AP: Frame: QUAD/X
AP: Barometer 1 calibration complete
AP: Barometer 2 calibration complete
AP: Ready
AP: ArduPilot Ready
AP: AHRS: DCM active
Fence present
AP: RCT: SaveWaypoint LOW
AP: RCR: Precision enter MIDDLE
Flight battery 100 percent
AP: EXF3 IMU0 initialised
AP: EXF3 IMU1 initialised
AP: AHRS: EXF3 active
AP: EXF3 IMU0 tilt alignment complete
AP: EXF3 IMU1 tilt alignment complete
AP: EXF3 IMU0 MAG0 initial yaw alignment complete
AP: EXF3 IMU1 MAG0 initial yaw alignment complete

```

Integrating with Drone Operations

Data Acquisition and Control

Sensor Data Use the UART connection to receive telemetry data from the APM 2.8, such as GPS coordinates, altitude, and battery status. **Control Commands** Send commands from the Raspberry Pi to the APM 2.8 to control the drone's movement, such as changing waypoints, adjusting altitude, or initiating a return-to-home sequence.

Advanced Processing

Use the Raspberry Pi to process the data in real-time, applying computer vision algorithms for fire detection, thermal imaging analysis, and decision-making.

Implement machine learning models to predict fire spread and optimize drone deployment for firefighting efforts.

SOFTWARE SETUP:

Raspberry Pi OS Installation Install the Raspberry Pi OS on the Raspberry Pi board.
MAVLink Library Installation Install the MAVLink library on the Raspberry Pi to facilitate communication with the APM 2.8 Autopilot.
Python Development Environment Set up a Python development environment on the Raspberry Pi for writing scripts to interact with the APM 2.8. For the firefighter drone project, configuring the software environment on the Raspberry Pi 4 is crucial. This setup involves installing the Raspberry Pi OS, the MAVLink library, and setting up a Python development environment. These steps ensure seamless communication and control between the Raspberry Pi and the APM 2.8 flight controller.

Raspberry Pi OS Installation

Step 1: Download Raspberry Pi OS

- Visit the official Raspberry Pi website and download the latest version of Raspberry Pi OS (formerly known as Raspbian). Choose the "Raspberry Pi OS with desktop" for a full-featured environment or "Raspberry Pi OS Lite" for a minimal installation.

Step 2: Flash Raspberry Pi OS to SD Card

- Use a tool like Raspberry Pi Imager or balenaEtcher to flash the downloaded OS image to a microSD card.
 1. Insert the microSD card into your computer.
 2. Open Raspberry Pi Imager or balenaEtcher.
 3. Select the downloaded OS image.
 4. Choose the microSD card as the target device.
 5. Click "Write" to start the flashing process.

Step 3: Initial Setup

- Insert the microSD card into the Raspberry Pi 4 and power it on.
- Follow the on-screen prompts to complete the initial setup, including configuring the keyboard, language, and network settings.
- Update the system packages by opening a terminal and running:

CHAPTER 7

CONCLUSION AND FUTUR WORK

CHAPTER 7: CONCLUSION

7.1. Conclusion

The Fire Flight drone project showcases the transformative potential of drone technology in combating wildfires. It emphasizes improved safety, efficiency, and cost-effectiveness, contributing to better wildfire management and environmental conservation. Human and explosives detection using cameras can help the firemen find distressed civilians. Using Fire Extinguisher Ball, fire in a particular area can be reduced and an entry or exit route can be created. The toxicity reports generated by the gas sensors can also help the firemen protect themselves. The cameras, sensors and other hardware used to make the drone are cost effective so that the drone would be affordable for the Fire-Fighter Department

Despite the significant advantages of using drones in wildfire management, there are several limitations and challenges associated with Fire Flight control:

Weather Conditions:

- Adverse weather conditions, such as high winds, heavy rain, or smoke, can hinder drone operations.
- Strong winds can make it difficult for drones to maintain stability and control, potentially leading to crashes.

Battery Life and Flight Duration:

- Most drones have limited battery life, restricting their operational time.
- Frequent battery changes or recharging can interrupt continuous surveillance and monitoring efforts.

Range and Coverage:

- Drones have a limited operational range, which can be a constraint in large wildfire scenarios.
- Multiple drones might be required to cover extensive areas, complicating coordination and control.

Data Transmission and Communication:

- Reliable communication networks are essential for transmitting real-time data from drones to ground control.
- Remote or mountainous areas might have poor connectivity, affecting data transmission and control signals.

7.2. Future Work

Extended Battery Life and Flight Duration:

- Development of more efficient battery technologies or alternative power sources (e.g., solar power) to extend flight times.
- Incorporation of rapid charging systems and battery swap mechanisms to minimize downtime.

Enhanced Weather Resistance:

- Designing drones that can operate effectively in various weather conditions, including high winds, rain, and smoke.
- Implementing advanced stabilization and navigation systems to maintain control in adverse conditions.

Increased Range and Coverage:

- Development of long-range drones capable of covering larger areas without frequent returns to base.
- Implementation of drone swarming technology to coordinate multiple drones for comprehensive area coverage.

Improved Data Transmission and Communication:

- Enhancing communication systems to ensure reliable data transmission in remote or rugged terrain.
- Utilizing satellite communication links or mesh networks to maintain connectivity over vast distances.

REFERENCES

REFERENCES

- [1] [iq_tutorials/docs/Installing_Ardupilot.md at master · Intelligent-Quads/iq_tutorials · GitHub](#)
- [2] [iq_tutorials/docs/installing_qgc.md at master · Intelligent-Quads/iq_tutorials · GitHub](#)
- [3] [iq_tutorials/docs/installing_gazebo_arduplugin.md at master · Intelligent-Quads/iq_tutorials · GitHub](#)
- [4] <https://www.raspberrypi.com/documentation/>
- [5] <https://ardupilot.org/>
- [6] <https://mavlink.io/en/>
- [7] <https://ardupilot.org/mavproxy/>
- [8] <https://github.com/ArduPilot/pymavlink>
- [9] <https://ardupilot.org/dev/docs/raspberry-pi-via-mavlink.html>
- [10] https://docs.opencv.org/4.x/d6/d00/tutorial_py_root.html
- [11] <https://www.datacamp.com/blog/yolo-object-detection-explained>
- [12] <https://www.geeksforgeeks.org/opencv-overview/>
- [13] <https://usawire.com/understanding-8-basic-points-about-3s-lipo-batteries/#:~:text=The%203S%20Lipo%20battery%20is,nominal%20value%20of%203.7%20volts>
- [14] [iq_tutorials/docs/installing_ros.md at master · Intelligent-Quads/iq_tutorials · GitHub](#)
- [15] [iq_tutorials/docs/gnc_tutorial.md at master · Intelligent-Quads/iq_tutorials · GitHub](#)

المخلص

الملخص

يتمثل الهدف الرئيسي لهذا المشروع في اقتراح تصميم وتطوير نظام طائرات بدون طيار للكشف عن الحرائق ومراقبتها للمساعدة في الكشف المبكر عن الحرائق وتقييمها والاستجابة لها. يتألف النظام المقترح من طائرة بدون طيار مصممة خصيصًا ومجهزة بكاميرا عالية الدقة، واختياريًا، كاميرا تصوير حراري لتعزيز قدرات الكشف عن الحرائق. تم دمج الطائرة بدون طيار مع وحدة GPS للملاحة المستقلة وتتبع الموقع.

بالإضافة إلى ذلك، تم تجهيز الطائرة بدون طيار بوحدة اتصال لنقل البيانات في الوقت الفعلي إلى محطة التحكم الأرضية. يتضمن المكون البرمجي للنظام خوارزميات الرؤية الحاسوبية لمعالجة الصور والكشف عن الحرائق. تحلل هذه الخوارزميات موجزات الفيديو الحية من كاميرات الطائرة بدون طيار للكشف عن علامات الحريق، مثل اللهب. عند اكتشاف حريق، يمكن للنظام تنبيه المستجيبين للطوارئ وتزويدهم ببيانات في الوقت الفعلي، بما في ذلك موقع الحريق وحجمه. سيتم تقييم فعالية النظام وأدائه من خلال المحاكاة والاختبارات الواقعية.

يهدف المشروع إلى إثبات جدوى وفائدة استخدام الطائرات بدون طيار للكشف عن الحرائق ومراقبتها، مع تسليط الضوء على إمكاناتها لتحسين كفاءة وفعالية عمليات مكافحة الحرائق والاستجابة للطوارئ. وقد تم استخدام الطائرات بدون طيار بشكل متزايد في مكافحة الحرائق لتحسين سرعة الاستجابة والحد من المخاطر التي يتعرض لها رجال الإطفاء من البشر. ومع ذلك، هناك عدد قليل من الدراسات التي تدرس في نفس الوقت التنبؤ بانتشار الحرائق، وجدولة الطائرات بدون طيار، وتكوين الموظفين والإمدادات الداعمة